

# MailAlly Architecture

Readable Notes from the Complete Application Architecture Guide

Generated as a clean, long-form PDF for study, review, and technical walkthroughs. The source guide content is preserved and reformatted with readable spacing, tables, code blocks, section breaks, and page numbers.

# Study Map

- Part 1 - What Is MailAlly?
- Part 2 - Complete System at a Glance
- Part 3 - Technology Stack
- Part 4 - Complete Repository Structure
- Part 5 - Frontend Architecture
- Part 6 - Frontend User Journey
- Part 7 - Frontend <-> Backend Communication
- Part 8 - Backend Architecture
- Part 9 - Backend Request Lifecycle
- Part 10 - Database Architecture
- Part 11 - Database Query Flow
- Part 12 - JPA / Hibernate / ORM
- Part 13 - Authentication & Security
- Part 14 - User / Role / Permission Architecture
- Part 15 - Multi-Tenancy
- Part 16 - Contact Management
- Part 17 - Template System
- Part 18 - Campaign Architecture
- Part 19 - Email Sending Engine
- Part 20 - Async / Queue / Batch Processing
- Part 21 - Redis / Cache / Real-Time State
- Part 22 - Scheduler
- Part 23 - Real-Time Campaign Monitoring
- Part 24 - Email Tracking
- Part 25 - Analytics Architecture
- Part 26 - API Architecture
- Part 27 - Error Handling
- Part 28 - Configuration & Environment
- Part 29 - External Services
- Part 30 - Testing Architecture
- Part 31 - Deployment & DevOps
- Part 32 - Complete Application Data Flows
- Part 33 - Complete Architecture Diagram
- Part 34 - File-by-File Learning Map
- Part 35 - How the Application Was Built
- Part 36 - What I Need to Learn
- Part 37 - Beginner -> Professional -> Architect
- Part 38 - How to Explain MailAlly to an Expert

Part 39 - Expert Questions I Should Be Ready For

Part 40 - Architectural Weaknesses

Part 41 - Practical Code-Tracing Exercises

Part 42 - Final "I Understand MailAlly" Checklist

Part 43 - Final One-Page Architecture Summary

# MailAlly - Complete Application Architecture Guide

*Reverse-engineered from the actual codebase.*

*Every class name, file path, package name, endpoint, entity, and technology referenced in this document is real and exists in the MailAlly repository.*

*Where something is planned but not yet implemented, it is explicitly marked.*

# PART 1 - WHAT IS MAILALLY?

## Business Perspective

MailAlly is a **multi-tenant SaaS email marketing platform** that allows organizations to manage contacts, design email templates, create and execute bulk email campaigns, track delivery/engagement events, and visualize analytics - all from a single web application.

## Problem Being Solved

Small-to-medium businesses and agencies need an affordable, self-hosted alternative to Mailchimp, SendGrid, or HubSpot that provides full control over email infrastructure, contact data, and campaign analytics without paying per-email SaaS fees.

## Target Users

- Marketing teams at SMBs
- Digital marketing agencies (the landing page is branded for "Marcamor," suggesting use by a specific agency)
- Business owners who want to send newsletters, promotional emails, and transactional campaigns

## Main Features (Actually Implemented)

- **Organization & User Management** - Multi-tenant isolation, user registration, role-based access
- **Contact Management** - Manual creation, CSV/Excel bulk import, Google Sheets/Drive import, collections, tags, dynamic fields, audit history, timeline
- **Email Template Builder** - HTML templates with `{{variable}}` personalization, AI-powered template generation (Gemini)
- **Campaign Engine** - Create campaigns, attach templates and segments, launch with batch processing, real-time progress monitoring
- **Multi-Provider Email Engine** - SMTP (Brevo relay), Brevo HTTP API, Amazon SES, Mock provider - with automatic failover and circuit breaker
- **Webhook Processing** - Brevo and SES webhook endpoints for delivery/bounce/open/click events
- **Analytics Dashboard** - Campaign analytics, provider analytics, audience analytics, time-series charts, CSV/Excel export
- **Scheduler** - Schedule campaigns for future execution
- **Billing & Subscriptions** - Usage tracking and subscription plan management
- **Notifications** - In-app notification system with read/unread tracking
- **Audit Logging** - System-wide activity audit trail
- **AI Assistant** - Gemini-powered subject line generation, content writing, spam scoring
- **Google Integration** - OAuth2 integration for Google Drive and Google Sheets contact import
- **Real-Time Monitoring** - WebSocket push for live analytics, SSE for campaign progress streaming

## SaaS Model

- Organizations sign up and get isolated data environments
- Each organization has a subscription plan (stored in subscriptions table)
- Plans define limits: `maxContacts`, `maxEmailsPerMonth`, `maxUsers`
- Default plan: "Enterprise Unlimited" (1M contacts, 5M emails/month, 50 users, \$99/month)
- [PARTIALLY IMPLEMENTED] - Billing records exist but no payment gateway integration

## Core Business Value

"Send the right email to the right person at the right time, track what happens, and learn from the results - all while keeping your data private and your costs predictable."

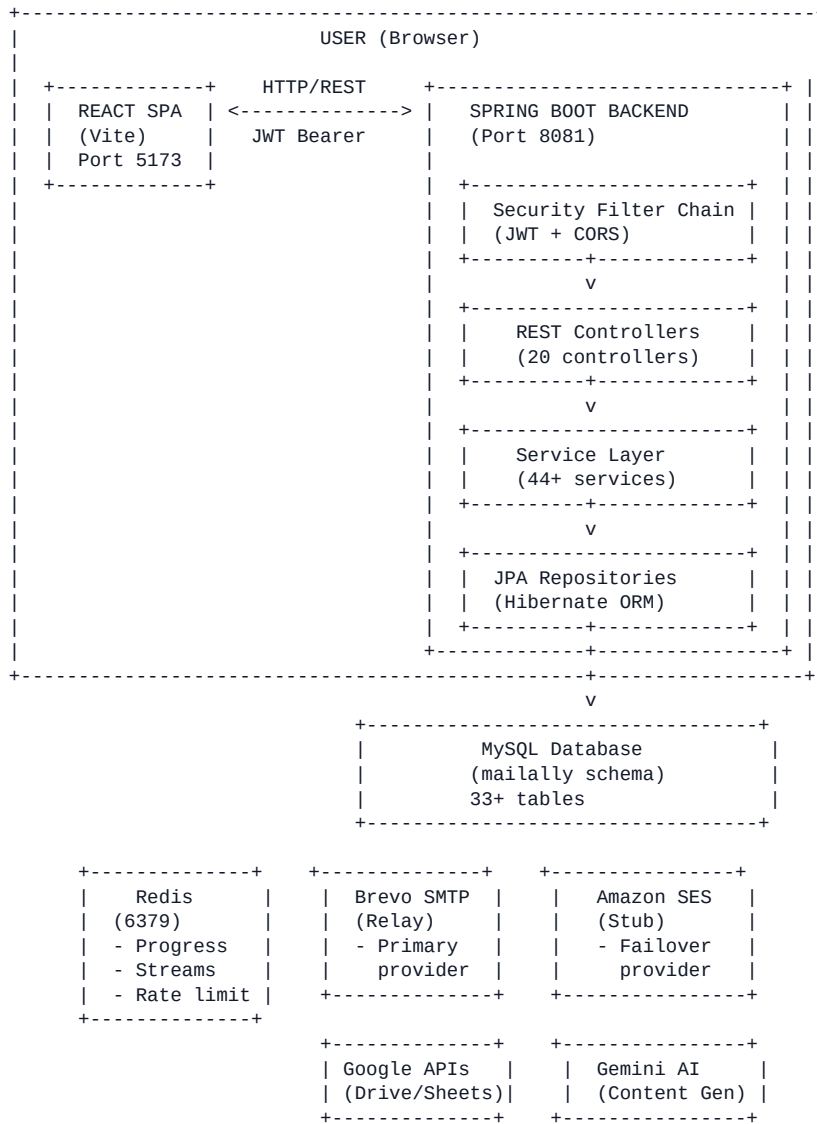
## Technical Perspective

MailAlly is a **full-stack web application** consisting of:

| Layer             | Technology                                | Actual Location                      |
|-------------------|-------------------------------------------|--------------------------------------|
| Frontend SPA      | React 19 + Vite + TailwindCSS 4           | mailally-frontend/                   |
| REST API Backend  | Spring Boot 4.0.7 + Java 21               | mailally-backend/                    |
| Database          | MySQL 8                                   | jdbc:mysql://localhost:3306/mailally |
| Cache / Queue     | Redis                                     | localhost:6379                       |
| Message Bus       | Apache Kafka                              | localhost:9092 [DISABLED by default] |
| Email Providers   | Brevo SMTP, Brevo HTTP API, AWS SES, SMTP | External services                    |
| AI Engine         | Google Gemini API                         | External service                     |
| File Processing   | Apache POI (Excel), Commons CSV           | Bundled libraries                    |
| Authentication    | JWT (jjwt 0.12.6) + BCrypt                | Stateless                            |
| API Documentation | SpringDoc OpenAPI (Swagger UI)            | /swagger-ui/**                       |
| Monitoring        | Micrometer + Prometheus                   | /actuator/prometheus                 |
| Scheduling        | Spring Quartz (in-memory store)           | spring.quartz.job-store-type=memory  |

# PART 2 - COMPLETE SYSTEM AT A GLANCE

## Architecture Diagram



## Component Table

| Component          | Technology                            | Purpose                                          | Actual Location               | Communicates With                 |
|--------------------|---------------------------------------|--------------------------------------------------|-------------------------------|-----------------------------------|
| Frontend SPA       | React 19 + Vite                       | User interface, forms, dashboards, charts        | mailally-frontend/src/        | Backend REST API via Axios        |
| API Gateway        | Spring Boot Controllers               | Request routing, validation, response formatting | com.mailally.*.controller     | Services, DTOs                    |
| Security Layer     | Spring Security + JWT                 | Authentication, authorization, CORS              | com.mailally.security         | Controllers, UserDetails          |
| Auth Module        | AuthService + AuthController          | Registration, login, password management         | com.mailally.auth             | Users, Organizations, JWT         |
| Contact Module     | ContactService + Controller           | CRUD, import, collections, timeline              | com.mailally.contact          | Database, Google Integration      |
| Template Module    | TemplateService + Controller          | Template CRUD, variable engine, AI generation    | com.mailally.template         | Database, AI Service              |
| Campaign Module    | CampaignService + Controller          | Campaign lifecycle management                    | com.mailally.campaign         | Templates, Segments, Email Engine |
| Email Engine       | EmailService + Workers + Orchestrator | Email dispatch, batching, failover, retries      | com.mailally.email            | Providers, Redis, Database        |
| Provider Layer     | EmailProviderFactory + Providers      | SMTP, Brevo API, SES abstraction                 | com.mailally.email.provider   | External SMTP/API services        |
| Redis Cache        | RedisProgressCache + RedisRateLimiter | Fast counters, streams, rate limiting            | com.mailally.email.cache      | CampaignWorkerService             |
| Webhook Handler    | WebhookController + ResolverService   | Process delivery events from providers           | com.mailally.email.controller | EmailEvent repository, WebSocket  |
| Analytics Engine   | AnalyticsService + EventAggregation   | Metrics, charts, reports, exports                | com.mailally.analytics        | All repositories, providers       |
| Dashboard          | DashboardService + Controller         | Executive KPIs, system health, quick actions     | com.mailally.dashboard        | All repositories                  |
| Scheduler          | SchedulerService + Quartz             | Campaign scheduling                              | com.mailally.scheduler        | Campaigns, Quartz                 |
| Notification       | NotificationService + WebSocket       | In-app alerts                                    | com.mailally.notification     | Users, WebSocket                  |
| Billing            | BillingService + Controller           | Usage tracking, invoices                         | com.mailally.billing          | Subscriptions                     |
| Audit              | AuditService + Controller             | Activity logging                                 | com.mailally.audit            | All modules                       |
| AI                 | AIService + Controller                | Content generation, spam scoring                 | com.mailally.ai               | Gemini API                        |
| Google Integration | GoogleOAuth + Drive + Sheets          | Contact import from Google ecosystem             | com.mailally.integration      | Google APIs                       |
| Settings           | SettingsService + Controller          | Application configuration                        | com.mailally.settings         | Database                          |
| Subscription       | SubscriptionService + Controller      | Plan management, quotas                          | com.mailally.subscription     | Organizations                     |



## PART 3 - TECHNOLOGY STACK

| Layer            | Technology                               | Version          | Why Used                                            | Where Used                          |
|------------------|------------------------------------------|------------------|-----------------------------------------------------|-------------------------------------|
| Language         | Java                                     | 21               | LTS with virtual threads, pattern matching          | Backend                             |
| Framework        | Spring Boot                              | 4.0.7            | Enterprise-grade, dependency injection, auto-config | Backend                             |
| Web              | Spring MVC                               | (Boot-managed)   | REST controllers, request mapping                   | Controllers                         |
| Security         | Spring Security                          | (Boot-managed)   | Authentication, authorization, filter chain         | com.mailally.security               |
| ORM              | Spring Data JPA + Hibernate              | (Boot-managed)   | Database abstraction, entity management             | Repositories                        |
| Database         | MySQL                                    | 8.x              | Relational data, ACID transactions                  | localhost:3306/mailally             |
| Cache            | Redis                                    | Latest           | Fast counters, stream queues, rate limiting         | localhost:6379                      |
| Message Broker   | Apache Kafka                             | Latest           | Event-driven architecture (disabled by default)     | KafkaConfig.java                    |
| JWT              | jjwt (io.jsonwebtoken)                   | 0.12.6           | Stateless authentication tokens                     | JwtService.java                     |
| Password Hashing | BCrypt                                   | (Spring-managed) | Secure password storage                             | SecurityConfig.java                 |
| Validation       | Jakarta Validation (Hibernate Validator) | (Boot-managed)   | DTO field validation, @Valid                        | DTOs                                |
| JSON             | Jackson                                  | (Boot-managed)   | JSON serialization/deserialization                  | All API communication               |
| Email (SMTP)     | Spring Mail (JavaMailSender)             | (Boot-managed)   | SMTP email dispatch                                 | SmtEmailProvider.java               |
| Email (API)      | Brevo REST API v3                        | Latest           | HTTP-based bulk email dispatch                      | BrevoEmailProvider.java             |
| Email (Cloud)    | AWS SES                                  | -                | Cloud email service [STUB]                          | SesEmailProvider.java               |
| File Processing  | Apache POI                               | 5.3.0            | Excel (.xlsx) file parsing for contact import       | ContactServiceImpl.java             |
| File Processing  | Apache Commons CSV                       | 1.11.0           | CSV file parsing for contact import                 | ContactServiceImpl.java             |
| Google APIs      | Google API Client + Drive + Sheets       | 2.7.0            | OAuth2, Google Drive file listing, Sheets reading   | com.mailally.integration            |
| AI               | Google Gemini API                        | -                | Content generation, spam scoring                    | AIServiceImpl.java                  |
| WebSocket        | Spring WebSocket (STOMP)                 | (Boot-managed)   | Real-time analytics push to frontend                | WebSocketConfig.java                |
| SSE              | Spring SseEmitter                        | (Boot-managed)   | Live campaign progress streaming                    | CampaignAsyncExecutor.java          |
| Scheduler        | Spring Quartz                            | (Boot-managed)   | Timed campaign execution                            | spring.quartz.job-store-type=memory |
| Monitoring       | Micrometer + Prometheus                  | (Boot-managed)   | Application metrics export                          | ObservabilityConfig.java            |
| API Docs         | SpringDoc OpenAPI                        | 2.8.5            | Swagger UI for API documentation                    | /swagger-ui/**                      |
| Boilerplate      | Lombok                                   | (Boot-managed)   | Getter/setter/builder generation                    | Entities, DTOs                      |

| Layer              | Technology           | Version        | Why Used                               | Where Used         |
|--------------------|----------------------|----------------|----------------------------------------|--------------------|
| <b>Build</b>       | Maven                | -              | Dependency management, build lifecycle | pom.xml            |
| <b>Dev Tools</b>   | Spring Boot DevTools | (Boot-managed) | Hot reload during development          | Runtime            |
| <b>Frontend</b>    | React                | 19.0.0         | Component-based UI framework           | mailally-frontend/ |
| <b>Build Tool</b>  | Vite                 | 6.0.7          | Fast frontend build and dev server     | vite.config.js     |
| <b>Routing</b>     | React Router DOM     | 7.1.3          | Client-side page routing               | AppRoutes.jsx      |
| <b>HTTP Client</b> | Axios                | 1.7.9          | HTTP requests with interceptors        | axiosClient.js     |
| <b>CSS</b>         | TailwindCSS          | 4.0.0          | Utility-first CSS framework            | index.css          |
| <b>Charts</b>      | Recharts             | 2.15.0         | Data visualization (bar, line, pie)    | Analytics pages    |
| <b>Icons</b>       | Lucide React         | 0.474.0        | SVG icon library                       | All pages          |

## PART 4 - COMPLETE REPOSITORY STRUCTURE

```
mailally-backend/                                <- Root project directory
+-- mailally-backend/                            <- Spring Boot backend
|   +-- pom.xml                                  <- Maven dependencies
|   +-- src/main/java/com/mailally/
|   |   +-- MailallyBackendApplication.java <- Entry point + startup repair
|   |   +-- ai/                                <- AI content generation module
|   |   |   +-- controller/AiController.java
|   |   |   +-- dto/                            <- AI request/response DTOs
|   |   |   +-- entity/Ai.java                 <- AI usage tracking entity
|   |   |   +-- mapper/                        <- Entity<->DTO mapping
|   |   |   +-- provider/                      <- Gemini API provider
|   |   |   +-- repository/                   <- JPA repository
|   |   |   +-- service/impl/AIServiceImpl.java
|   |   |   +-- validator/
|   |   +-- analytics/                        <- Analytics & reporting module
|   |   |   +-- calculator/                   <- Metric calculation logic
|   |   |   +-- controller/AnalyticsController.java
|   |   |   +-- dto/                          <- 12+ analytics DTOs
|   |   |   +-- engine/                      <- EventAggregationEngine
|   |   |   +-- entity/Analytics.java
|   |   |   +-- mapper/AnalyticsMapper.java
|   |   |   +-- provider/                    <- Analytics data providers
|   |   |   +-- repository/
|   |   |   +-- service/impl/AnalyticsServiceImpl.java
|   |   |   +-- validator/
|   |   +-- audit/                          <- Audit logging module
|   |   |   +-- controller/AuditController.java
|   |   |   +-- entity/Audit.java
|   |   |   +-- service/impl/AuditServiceImpl.java
|   |   +-- auth/                          <- Authentication module
|   |   |   +-- controller/AuthController.java
|   |   |   +-- dto/ (LoginRequestDto, RegisterRequestDto, AuthResponseDto, etc.)
|   |   |   +-- entity/Auth.java
|   |   |   +-- mapper/AuthMapper.java
|   |   |   +-- repository/AuthRepository.java
|   |   |   +-- service/impl/AuthServiceImpl.java
|   |   |   +-- validator/AuthValidator.java
|   |   +-- billing/                        <- Billing module
|   |   |   +-- controller/BillingController.java
|   |   |   +-- entity/Billing.java
|   |   |   +-- gateway/                    <- Payment gateway abstraction
|   |   |   +-- service/impl/BillingServiceImpl.java
|   |   +-- campaign/                      <- Campaign management module
|   |   |   +-- controller/CampaignController.java
|   |   |   +-- dto/
|   |   |   +-- entity/Campaign.java, CampaignRecipient.java, CampaignActivityLog.java
|   |   |   +-- mapper/
|   |   |   +-- repository/CampaignRepository.java
|   |   |   +-- service/ (CampaignServiceImpl, CampaignBatchProcessor,
CampaignDiagnosticsService)
|   |   |   +-- validator/
|   |   +-- common/                        <- Shared types
|   |   |   +-- constants/
|   |   |   +-- enums/
|   |   |   +-- request/
|   |   |   +-- response/ApiResponse.java <- Standard API wrapper
|   |   |   +-- config/                    <- Application configuration
|   |   |   |   +-- AppConfig.java
|   |   |   |   +-- AsyncConfig.java        <- Thread pool for async campaigns
|   |   |   |   +-- CorsConfig.java
|   |   |   |   +-- DefaultDataInitializer.java <- Seed default users on startup
|   |   |   |   +-- JacksonConfig.java
|   |   |   |   +-- KafkaConfig.java        <- Kafka producer/consumer (disabled)
|   |   |   |   +-- ObservabilityConfig.java <- Prometheus metrics
|   |   |   |   +-- SwaggerConfig.java
|   |   |   |   +-- WebSocketConfig.java    <- STOMP WebSocket config
|   |   |   +-- constant/                  <- Application-wide constants
|   |   |   +-- contact/                  <- Contact management module
|   |   |   |   +-- controller/ContactController.java
|   |   |   |   +-- dto/
|   |   |   |   +-- entity/ (Contact, Tag, ImportBatch, ImportError, SavedFilter,
|   |   |   |   |   DynamicFieldRegistry, ContactTimeline, ContactCollection,
|   |   |   |   |   ContactAuditHistory)
|   |   |   |   +-- mapper/
|   |   |   |   +-- pipeline/              <- Import processing pipeline
|   |   |   |   +-- provider/              <- Import source providers
```

```

| | | +-- repository/
| | | +-- service/ (ContactServiceImpl, GoogleDriveImportService)
| | | +-- validator/
| | | +-- dashboard/ <- Executive dashboard module
| | | | +-- controller/DashboardController.java
| | | | +-- entity/Dashboard.java
| | | | +-- service/impl/DashboardServiceImpl.java
| | | +-- email/ <- EMAIL ENGINE (core module)
| | | | +-- cache/RedisProgressCache.java
| | | | +-- config/EmailEngineConfig.java
| | | | +-- constant/EmailEventType.java
| | | | +-- consumer/ (AnalyticsConsumer, AuditConsumer, BillingConsumer)
| | | | +-- controller/ (EmailController, WebhookController)
| | | | +-- dto/ (SendEmailRequestDto, LaunchCampaignRequestDto, CampaignProgressDto, etc.)
| | | | +-- entity/ (Email, EmailEvent, EmailQueue, CampaignBatch,
| | | | | CampaignRecipientLog, UnresolvedWebhookEvent)
| | | | +-- event/KafkaEventPublisher.java
| | | | +-- mapper/EmailMapper.java
| | | | +-- orchestrator/ (CampaignOrchestrator, BatchGenerator)
| | | | +-- provider/ (EmailProvider interface, EmailProviderFactory,
| | | | | SmtEmailProvider, BrevoEmailProvider, SesEmailProvider,
| | | | | MockEmailProvider, ProviderCircuitBreaker, ProviderHealthService)
| | | | +-- queue/ (RetryEngine, RedisRateLimiter)
| | | | +-- renderer/TemplateRenderer.java
| | | | +-- repository/ (EmailRepository, EmailQueueRepository,
| | | | | EmailEventRepository, CampaignBatchRepository,
| | | | | CampaignRecipientLogRepository)
| | | | +-- scheduler/ProgressSyncScheduler.java
| | | | +-- service/ (EmailService, EmailServiceImpl, CampaignAsyncExecutor,
| | | | | EmailEngineServiceImpl, WebhookResolverServiceImpl, EventNormalizer)
| | | | +-- validator/ (EmailValidator, WebhookValidator)
| | | | +-- worker/CampaignWorkerService.java
| | | +-- exception/
| | | | +-- CustomException.java
| | | | +-- GlobalExceptionHandler.java
| | | +-- integration/ <- Google OAuth/Drive/Sheets
| | | | +-- controller/GoogleIntegrationController.java
| | | | +-- entity/GoogleIntegration.java
| | | | +-- service/ (GoogleAuthService, GoogleDriveService,
| | | | | GoogleSheetsService, TokenEncryptionService)
| | | +-- notification/ <- Notification module
| | | | +-- channel/ <- Notification channel abstraction
| | | | +-- controller/NotificationController.java
| | | | +-- entity/Notification.java
| | | | +-- service/impl/NotificationServiceImpl.java
| | | +-- organization/ <- Organization/tenant module
| | | | +-- controller/OrganizationController.java
| | | | +-- entity/Organization.java
| | | | +-- service/impl/OrganizationServiceImpl.java
| | | +-- scheduler/ <- Campaign scheduler module
| | | | +-- controller/SchedulerController.java
| | | | +-- entity/Scheduler.java
| | | | +-- service/impl/SchedulerServiceImpl.java
| | | +-- security/ <- Security infrastructure
| | | | +-- SecurityConfig.java <- Filter chain + CORS + BCrypt
| | | | +-- JwtService.java <- Token generation/validation
| | | | +-- JwtAuthenticationFilter.java <- Request filter
| | | | +-- CustomUserDetails.java <- UserDetails implementation
| | | | +-- CustomUserDetailsService.java
| | | | +-- CustomAuthenticationEntryPoint.java
| | | +-- segment/ <- Contact segmentation module
| | | | +-- controller/SegmentController.java
| | | | +-- entity/Segment.java
| | | | +-- service/impl/SegmentServiceImpl.java
| | | +-- settings/ <- Application settings module
| | | | +-- controller/SettingsController.java
| | | | +-- entity/Settings.java
| | | | +-- service/impl/SettingsServiceImpl.java
| | | +-- subscription/ <- Subscription plan module
| | | | +-- controller/SubscriptionController.java
| | | | +-- entity/Subscription.java
| | | | +-- service/impl/SubscriptionServiceImpl.java
| | | +-- template/ <- Email template module
| | | | +-- controller/TemplateController.java
| | | | +-- entity/Template.java
| | | | +-- service/ (TemplateServiceImpl, TemplateVariableEngine, AiTemplateService)
| | | | +-- validator/
| | | +-- user/ <- User management module
| | | | +-- controller/UserController.java
| | | | +-- entity/User.java

```

```

| | | +-- service/impl/UserServiceImpl.java
| | +-- util/ <- Utilities
| | | +-- DateUtil.java
| | | +-- StringUtil.java
| | | +-- ValidationUtil.java
| +-- src/main/resources/
| | +-- application.properties <- All configuration
| | +-- billing_schema.sql
| | +-- contact_enterprise_schema.sql
| | +-- email_engine_schema.sql
| | +-- enterprise_workflow_schema.sql
| | +-- notification_schema.sql
| | +-- scheduler_schema.sql
| | +-- settings_schema.sql
| | +-- support_modules_schema.sql
|
+-- mailally-frontend/ <- React frontend
| +-- package.json
| +-- vite.config.js
| +-- index.html
| +-- src/
| | +-- main.jsx <- React entry point
| | +-- App.jsx <- Root component with providers
| | +-- index.css <- Global styles (TailwindCSS)
| | +-- api/ <- API client layer
| | | +-- axiosClient.js <- Axios instance + JWT interceptor
| | | +-- authApi.js
| | | +-- contactApi.js
| | | +-- campaignApi.js <- Also exports templateApi, segmentApi, schedulerApi
| | | +-- dashboardApi.js
| | | +-- extraApis.js <- analyticsApi, notificationApi, settingsApi,
| | | billingApi, etc.
| | +-- context/
| | | +-- AuthContext.jsx <- Authentication state provider
| | | +-- ThemeContext.jsx <- Dark/light theme provider
| | +-- routes/
| | | +-- AppRoutes.jsx <- All application routes
| | +-- components/
| | | +-- common/Toast.jsx <- Toast notification system
| | | +-- layout/MainLayout.jsx <- Sidebar + top bar layout
| | | +-- campaigns/ <- Campaign-specific components
| | | +-- contacts/ <- Contact-specific components
| | | +-- templates/ <- Template-specific components
| | +-- pages/
| | | +-- AgencyLandingPage.jsx <- Public landing page (93KB)
| | | +-- NotFoundPage.jsx
| | | +-- auth/ (LoginPage, RegisterPage)
| | | +-- dashboard/ExecutiveDashboardPage.jsx
| | | +-- contacts/ContactsPage.jsx
| | | +-- templates/TemplatesPage.jsx
| | | +-- campaigns/ (CampaignsPage, CampaignWizardPage, CampaignAnalyticsPage)
| | | +-- analytics/AnalyticsPage.jsx
| | | +-- scheduler/SchedulerPage.jsx
| | | +-- notifications/NotificationsPage.jsx
| | | +-- settings/SettingsPage.jsx
| | | +-- billing/BillingPage.jsx
| | | +-- subscriptions/SubscriptionsPage.jsx
| | | +-- audit/AuditLogsPage.jsx
| | | +-- ai/AiAssistantPage.jsx
| | | +-- users/UsersPage.jsx
| | +-- utils/
| | | +-- analyticsFormatters.js
|
+-- docs/ <- Documentation
+-- images/ <- Asset images
+-- start_app.bat / start_app.ps1 <- Launch scripts
+-- blueprint.md <- Original design blueprint
+-- README.md
+-- clean_project.bat

```

# PART 5 - FRONTEND ARCHITECTURE

## Overview

The frontend is a **React 19 Single Page Application (SPA)** built with Vite 6, styled with TailwindCSS 4, using React Router DOM 7 for client-side routing, Axios for API communication, Recharts for data visualization, and Lucide React for icons.

## Entry Point

**File:** mailally-frontend/src/main.jsx

React.StrictMode -> App component

**File:** mailally-frontend/src/App.jsx

```
BrowserRouter
+- AuthProvider      <- Authentication context (token + user state)
  +- ThemeProvider   <- Dark/light mode toggle
    +- ToastProvider <- Notification toast system
      +- AppRoutes <- All page routes
```

## Routing (AppRoutes.jsx)

| Route                    | Page Component                                            | Auth Required |
|--------------------------|-----------------------------------------------------------|---------------|
| /                        | AgencyLandingPage (or redirect to dashboard if logged in) | No            |
| /landing                 | AgencyLandingPage                                         | No            |
| /login                   | LoginPage                                                 | No            |
| /register                | RegisterPage                                              | No            |
| /dashboard               | ExecutiveDashboardPage                                    | Yes           |
| /contacts                | ContactsPage                                              | Yes           |
| /templates               | TemplatesPage                                             | Yes           |
| /campaigns               | CampaignsPage                                             | Yes           |
| /campaigns/wizard        | CampaignWizardPage                                        | Yes           |
| /campaigns/:id/analytics | CampaignAnalyticsPage                                     | Yes           |
| /analytics               | AnalyticsPage                                             | Yes           |
| /scheduler               | SchedulerPage                                             | Yes           |
| /notifications           | NotificationsPage                                         | Yes           |
| /settings                | SettingsPage                                              | Yes           |
| /billing                 | BillingPage                                               | Yes           |
| /subscriptions           | SubscriptionsPage                                         | Yes           |
| /audit                   | AuditLogsPage                                             | Yes           |
| /ai                      | AiAssistantPage                                           | Yes           |
| /users                   | UsersPage                                                 | Yes           |
| *                        | NotFoundPage                                              | No            |

## Authentication State (AuthContext.jsx)

- Token stored in localStorage as mailally\_token
- User object stored as mailally\_user
- On login: sends POST to /api/v1/auth/login, stores JWT and user data

- On 401 response: clears token and redirects to /login (via Axios interceptor)
- **Development fallback:** If backend is unreachable, creates mock token for any email/password combo with >=6 chars

## API Communication Layer (axiosClient.js)

- Base URL: http://localhost:8081/api/v1
- Request interceptor: Attaches Authorization: Bearer <token> header
- Response interceptor: On 401, clears storage and redirects to /login

## Data Flow Pattern

```

User Action (click button)
  v
Page Component (e.g., CampaignsPage)
  v
API Module (e.g., campaignApi.launchCampaign(id))
  v
Axios HTTP Request (POST /api/v1/campaigns/{id}/launch)
  v
JWT attached via interceptor
  v
Backend Controller processes request
  v
JSON Response
  v
Page component updates React state
  v
UI re-renders with new data

```

# PART 6 - FRONTEND USER JOURNEY

1. LANDING PAGE (/)
  - | <- AgencyLandingPage.jsx (93KB - full marketing site)
  - | <- Shows features, pricing, testimonials, CTA
  - v
2. REGISTRATION (/register)
  - | <- RegisterPage.jsx
  - | <- POST /api/v1/auth/register
  - | <- Creates: Organization + User + Auth + Subscription
  - v
3. LOGIN (/login)
  - | <- LoginPage.jsx
  - | <- POST /api/v1/auth/login
  - | <- Returns JWT token + user profile
  - | <- Token stored in localStorage
  - v
4. DASHBOARD (/dashboard)
  - | <- ExecutiveDashboardPage.jsx
  - | <- GET /api/v1/dashboard/overview -> KPIs, charts, health
  - | <- Shows: total campaigns, contacts, emails sent, delivery rate
  - v
5. CONTACTS (/contacts)
  - | <- ContactsPage.jsx
  - | <- GET /api/v1/contacts -> Paginated list
  - | <- POST /api/v1/contacts -> Create single contact
  - | <- POST /api/v1/contacts/import/start -> CSV/Excel upload
  - | <- Supports: collections, tags, filters, inline editing, timeline
  - v
6. TEMPLATES (/templates)
  - | <- TemplatesPage.jsx
  - | <- GET /api/v1/templates -> List templates
  - | <- POST /api/v1/templates -> Create with HTML + {{variables}}
  - | <- POST /api/v1/templates/generate-ai -> AI-generated template
  - v
7. CAMPAIGNS (/campaigns)
  - | <- CampaignsPage.jsx -> List all campaigns
  - | <- CampaignWizardPage.jsx -> Multi-step campaign creation
  - | <- POST /api/v1/campaigns -> Create campaign
  - | <- POST /api/v1/campaigns/{id}/attach-template/{templateId}
  - | <- POST /api/v1/campaigns/{id}/launch -> Start sending
  - v
8. LIVE MONITORING
  - | <- GET /api/v1/campaigns/{id}/live-progress
  - | <- SSE stream for real-time updates
  - | <- WebSocket /topic/analytics for event push
  - v
9. CAMPAIGN ANALYTICS (/campaigns/:id/analytics)
  - | <- CampaignAnalyticsPage.jsx
  - | <- GET /api/v1/analytics/campaigns/{id}
  - | <- Shows: sent/delivered/bounced/opened/clicked metrics
  - v
10. GLOBAL ANALYTICS (/analytics)
  - | <- AnalyticsPage.jsx
  - | <- GET /api/v1/analytics/dashboard -> Organization-wide metrics
  - v
11. SETTINGS (/settings)
  - | <- SettingsPage.jsx
  - | <- GET/PUT /api/v1/settings
  - v
12. BILLING (/billing)
  - <- BillingPage.jsx
  - <- GET /api/v1/billing/summary



# PART 7 - FRONTEND <-> BACKEND COMMUNICATION

## API Communication Map

### Authentication APIs

| Frontend File        | Endpoint              | Method | Backend Controller          | Backend Service              |
|----------------------|-----------------------|--------|-----------------------------|------------------------------|
| authApi.login()      | /api/v1/auth/login    | POST   | AuthController.login()      | AuthServiceImpl.login()      |
| authApi.register()   | /api/v1/auth/register | POST   | AuthController.register()   | AuthServiceImpl.register()   |
| authApi.getProfile() | /api/v1/auth/me       | GET    | AuthController.getProfile() | AuthServiceImpl.getProfile() |

### Contact APIs

| Frontend File                 | Endpoint                      | Method           | Backend Controller | DB Tables                               |
|-------------------------------|-------------------------------|------------------|--------------------|-----------------------------------------|
| contactApi.getContacts()      | /api/v1/contacts              | GET              | ContactController  | contacts                                |
| contactApi.createContact()    | /api/v1/contacts              | POST             | ContactController  | contacts                                |
| contactApi.startImport()      | /api/v1/contacts/import/start | POST (multipart) | ContactController  | contacts, import_batches, import_errors |
| contactApi.getCollections()   | /api/v1/contacts/collections  | GET              | ContactController  | contact_collections                     |
| contactApi.getDynamicFields() | /api/v1/contacts/fields       | GET              | ContactController  | dynamic_field_registry                  |
| contactApi.inlineCellEdit()   | /api/v1/contacts/{id}/cell    | PATCH            | ContactController  | contacts, contact_audit_history         |

### Campaign APIs

| Frontend File                  | Endpoint                             | Method | Backend Controller | DB Tables                                            |
|--------------------------------|--------------------------------------|--------|--------------------|------------------------------------------------------|
| campaignApi.getCampaigns()     | /api/v1/campaigns                    | GET    | CampaignController | campaigns                                            |
| campaignApi.createCampaign()   | /api/v1/campaigns                    | POST   | CampaignController | campaigns                                            |
| campaignApi.launchCampaign()   | /api/v1/campaigns/{id}/launch        | POST   | CampaignController | campaigns, campaign_batches, campaign_recipient_logs |
| campaignApi.getLiveProgress()  | /api/v1/campaigns/{id}/live-progress | GET    | CampaignController | Redis + DB                                           |
| campaignApi.getDeliveryStats() | /api/v1/emails/stats/{id}            | GET    | EmailController    | emails                                               |

### Dashboard APIs

| Frontend File                  | Endpoint                        | Method | Backend Service      |
|--------------------------------|---------------------------------|--------|----------------------|
| dashboardApi.getOverview()     | /api/v1/dashboard/overview      | GET    | DashboardServiceImpl |
| dashboardApi.getKpis()         | /api/v1/dashboard/kpis          | GET    | DashboardServiceImpl |
| dashboardApi.getCharts()       | /api/v1/dashboard/charts        | GET    | DashboardServiceImpl |
| dashboardApi.getSystemHealth() | /api/v1/dashboard/system-health | GET    | DashboardServiceImpl |

## Analytics APIs

| Frontend File                       | Endpoint                    | Method |
|-------------------------------------|-----------------------------|--------|
| analyticsApi.getDashboardSummary()  | /api/v1/analytics/dashboard | GET    |
| analyticsApi.getCampaignAnalytics() | /api/v1/analytics/campaigns | GET    |
| analyticsApi.getProviderAnalytics() | /api/v1/analytics/providers | GET    |
| analyticsApi.getChartData()         | /api/v1/analytics/charts    | GET    |

# PART 8 - BACKEND ARCHITECTURE

## Architectural Style: Feature-Based Layered Modular Monolith

The backend is organized as a **modular monolith** where each business domain (auth, contact, campaign, email, analytics, etc.) is a self-contained package with its own:

- `controller/` - REST endpoints
- `dto/` - Data Transfer Objects
- `entity/` - JPA entities
- `mapper/` - Entity<->DTO mapping
- `repository/` - Spring Data JPA interfaces
- `service/` - Business logic (interface + impl/)
- `validator/` - Input validation logic

### Why This Architecture?

- **Separation of concerns** - Each feature is isolated in its own package
- **Testability** - Services can be tested independently
- **Evolution path** - Individual modules can be extracted into microservices later
- **Team scalability** - Different developers can own different modules

## Modules (24 packages under com.mailally)

| Package      | Purpose                                    | Key Classes                                                                                       |
|--------------|--------------------------------------------|---------------------------------------------------------------------------------------------------|
| ai           | AI-powered content generation              | AiController, AiServiceImpl                                                                       |
| analytics    | Metrics, reports, exports                  | AnalyticsController, AnalyticsServiceImpl, EventAggregationEngine                                 |
| audit        | Activity audit trail                       | AuditController, AuditServiceImpl                                                                 |
| auth         | Registration, login, passwords             | AuthController, AuthServiceImpl                                                                   |
| billing      | Usage tracking, invoices                   | BillingController, BillingServiceImpl                                                             |
| campaign     | Campaign CRUD, diagnostics                 | CampaignController, CampaignServiceImpl, CampaignBatchProcessor                                   |
| common       | Shared types (ApiResponse, enums)          | ApiResponse, enums                                                                                |
| config       | Application configuration beans            | AsyncConfig, KafkaConfig, WebSocketConfig, DefaultDataInitializer                                 |
| constant     | Application & security constants           | ApplicationConstants, SecurityConstants                                                           |
| contact      | Contact CRUD, import, collections          | ContactController, ContactServiceImpl, GoogleDriveImportService                                   |
| dashboard    | Executive dashboard data                   | DashboardController, DashboardServiceImpl                                                         |
| email        | <b>Core email engine</b> (18 sub-packages) | EmailController, EmailServiceImpl, CampaignOrchestrator, CampaignWorkerService, WebhookController |
| exception    | Error handling                             | CustomException, GlobalExceptionHandler                                                           |
| integration  | Google OAuth2, Drive, Sheets               | GoogleIntegrationController, GoogleOAuthService                                                   |
| notification | In-app notifications                       | NotificationController, NotificationServiceImpl                                                   |
| organization | Organization/tenant management             | OrganizationController, OrganizationServiceImpl                                                   |
| scheduler    | Campaign scheduling                        | SchedulerController, SchedulerServiceImpl                                                         |
| security     | JWT, filters, user details                 | SecurityConfig, JwtService, JwtAuthenticationFilter                                               |
| segment      | Contact segmentation                       | SegmentController, SegmentServiceImpl                                                             |
| settings     | Application settings                       | SettingsController, SettingsServiceImpl                                                           |
| subscription | Subscription plans, quotas                 | SubscriptionController, SubscriptionServiceImpl                                                   |
| template     | Email templates, AI generation             | TemplateController, TemplateServiceImpl, TemplateVariableEngine                                   |
| user         | User management                            | UserController, UserServiceImpl                                                                   |
| util         | Utility helpers                            | DateUtil, StringUtil, ValidationUtil                                                              |

# PART 9 - BACKEND REQUEST LIFECYCLE

## Complete Request Flow

```
Browser HTTP Request
  v
Tomcat (embedded, port 8081)
  v
Spring Security Filter Chain
+- CorsFilter (allows localhost:5173)
+- JwtAuthenticationFilter
|   +- Extract "Authorization: Bearer <jwt>" header
|   +- JwtService.extractUsername(jwt) -> email
|   +- CustomUserDetailsService.loadUserByUsername(email)
|   +- JwtService.isTokenValid(jwt, userDetails)
|   +- Set SecurityContextHolder.authentication
+- (pass to controller if authenticated)
  v
@RestController method
+- @RequestBody -> DTO (deserialized by Jackson)
+- @Valid -> Jakarta Validation (MethodArgumentNotValidException on failure)
+- @AuthenticationPrincipal CustomUserDetails -> current user context
+- Calls service layer
  v
@Service method
+- Validator validates business rules
+- Repository performs database operations
+- Entity <-> DTO mapping via Mapper
+- Returns DTO
  v
Controller wraps in ApiResponse<T>
  v
ResponseEntity<ApiResponse<T>> -> JSON
  v
HTTP Response to browser
```

## Traced Example: Login Flow

1. Frontend: LoginPage.jsx -> authApi.login(email, password)
2. HTTP: POST /api/v1/auth/login { email, password }
3. Security: JwtAuthenticationFilter SKIPS (endpoint in permitAll list)
4. Controller: AuthController.login(@Valid LoginRequestDto)
5. Service: AuthServiceImpl.login(dto)
  - a. UserRepository.findByEmailAndIsDeletedFalse(email) -> User entity
  - b. AuthRepository.findByUser(user) -> Auth entity
  - c. PasswordEncoder.matches(password, auth.passwordHash) -> verify BCrypt
  - d. Check user.status == "ACTIVE"
  - e. auth.setLastLoginAt(now) -> update login timestamp
  - f. CustomUserDetails(user, passwordHash) -> create security principal
  - g. JwtService.generateToken(userDetails) -> JWT with claims:  
{ userId, organizationId, role, sub: email, exp: 24h }
  - h. AuthMapper.toAuthResponseDto(user, token) -> response DTO
6. Controller wraps in ApiResponse { success: true, data: { token, user } }
7. Frontend: Stores token in localStorage, redirects to /dashboard

## Traced Example: Campaign Launch

1. Frontend: `campaignApi.launchCampaign(id)`
2. HTTP: `POST /api/v1/campaigns/{id}/launch` (JWT in header)
3. Security: `JwtAuthenticationFilter` validates JWT -> sets `SecurityContext`
4. Controller: `CampaignController` -> delegates to `EmailServiceImpl`
5. `EmailServiceImpl.launchCampaign()`:
  - a. Validate user is ADMIN or MANAGER
  - b. `CampaignRepository.findByIdAndOrganizationIdAndIsDeletedFalse()`
  - c. Validate campaign has template, status is DRAFT
  - d. Set status -> "RUNNING"
  - e. `ContactRepository.findByOrganizationIdAndIsDeletedFalse()`
  - f. Filter contacts: SUBSCRIBED or ACTIVE status
  - g. For EACH contact:
    - `TemplateRenderer.render(subject, contact)` -> personalized subject
    - `TemplateRenderer.render(htmlContent, contact)` -> personalized body
    - `EmailProviderFactory.sendWithFailover()` -> send email
    - Save Email record (delivery log)
    - Save `CampaignRecipientLog` (recipient tracking)
    - Save `EmailEvent` (immutable event log)
    - Save `EmailQueue` (queue record)
  - h. Update campaign: `totalRecipients, sentCount, failedCount`
  - i. Set status -> "COMPLETED"
6. Return `CampaignProgressDto`

# PART 10 - DATABASE ARCHITECTURE

## Database Configuration

- **Engine:** MySQL 8
- **Database name:** mailally
- **Connection:** jdbc:mysql://localhost:3306/mailally
- **DDL Strategy:** spring.jpa.hibernate.ddl-auto=update (Hibernate auto-creates/updates tables)
- **Batch size:** 500 (JDBC batching for bulk inserts)

## Entity/Table Mapping (33 entities)

| Entity Class           | Table Name                  | Key Columns                                                                                                                                                                                                         |
|------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Organization           | organizations               | id, subscription_id, name, slug, domain, status                                                                                                                                                                     |
| User                   | users                       | id, organization_id, first_name, last_name, email, role, status                                                                                                                                                     |
| Auth                   | (auth table)                | id, user_id, password_hash, mfa_enabled, reset_token, last_login_at                                                                                                                                                 |
| Subscription           | (subscriptions)             | id, name, code, price, currency, max_contacts, max_emails_per_month, max_users                                                                                                                                      |
| Contact                | contacts                    | id, organization_id, email, first_name, last_name, phone, company, department, designation, city, state, country, tags, custom_fields, source_type, collection_id                                                   |
| ContactCollection      | (contact_collections)       | id, organization_id, name                                                                                                                                                                                           |
| ContactTimeline        | (contact_timelines)         | id, contact_id, event description                                                                                                                                                                                   |
| ContactAuditHistory    | (contact_audit_history)     | id, contact_id, field, old_value, new_value                                                                                                                                                                         |
| Tag                    | (tags)                      | id, organization_id, name                                                                                                                                                                                           |
| SavedFilter            | (saved_filters)             | id, organization_id, name, filter_json                                                                                                                                                                              |
| ImportBatch            | (import_batches)            | id, organization_id, batch_code, file_name, total, success, failed                                                                                                                                                  |
| ImportError            | (import_errors)             | id, batch_id, row_number, error_message                                                                                                                                                                             |
| DynamicFieldRegistry   | (dynamic_field_registry)    | id, organization_id, field_name, field_type                                                                                                                                                                         |
| Template               | email_templates             | id, organization_id, name, subject, html_content, text_content, version, status                                                                                                                                     |
| Segment                | (segments)                  | id, organization_id, name, type (STATIC/DYNAMIC), filter_criteria, contact_count                                                                                                                                    |
| Campaign               | campaigns                   | id, organization_id, template_id, segment_id, name, subject, from_name, from_email, sender_name, sender_email, reply_to, status, scheduled_at, total_recipients, sent_count, failed_count                           |
| CampaignRecipient      | (campaign_recipients)       | id, campaign_id, contact_id [PLACEHOLDER]                                                                                                                                                                           |
| CampaignActivityLog    | (campaign_activity_logs)    | id, campaign_id, action, description                                                                                                                                                                                |
| Email                  | emails                      | id, organization_id, campaign_id, contact_id, recipient_email, subject, provider, status, response_id, error_message, retry_count, max_retries, sent_at, delivered_at, opened_at, clicked_at, bounced_at, failed_at |
| EmailEvent             | email_events                | id, organization_id, campaign_id, recipient_id, event_type (ENUM), provider, provider_message_id, user_agent, ip_address, metadata, occurred_at                                                                     |
| EmailQueue             | (email_queue)               | id, organization_id, campaign_id, contact_id, recipient_email, personalized_subject, personalized_html, provider, status, retry_count, batch_number                                                                 |
| CampaignBatch          | (campaign_batches)          | id, campaign_id, batch_number, optimal_size, status, worker_node_id, idempotency_key, started_at, completed_at, retry_count, provider_batch_id                                                                      |
| CampaignRecipientLog   | (campaign_recipient_logs)   | id, campaign_id, contact_id, email, status, provider, provider_message_id, attempts, last_error, smtp_response_code, worker_thread_id, duration_ms                                                                  |
| UnresolvedWebhookEvent | (unresolved_webhook_events) | id, provider, payload, attempts                                                                                                                                                                                     |





# PART 11 - DATABASE QUERY FLOW

```
Frontend Request (GET /api/v1/contacts?page=0&size=50)
  v
ContactController.getContacts(currentUser, pageable)
  v
ContactServiceImpl.getContacts(orgId, pageable)
  v
ContactRepository.findByOrganizationIdAndIsDeletedFalse(orgId, pageable)
  v
Spring Data JPA generates JPQL:
  SELECT c FROM Contact c WHERE c.organization.id = :orgId AND c.isDeleted = false
  v
Hibernate translates to SQL:
  SELECT * FROM contacts
  WHERE organization_id = ?
  AND is_deleted = 0
  ORDER BY created_at DESC
  LIMIT 50 OFFSET 0
  v
MySQL executes query -> ResultSet
  v
Hibernate maps rows -> Contact entities
  v
ContactMapper.toContactResponseDto(entity) -> DTO
  v
Page<ContactResponseDto> -> JSON -> HTTP Response
```

## Pagination

- Spring Data Pageable with PageRequest.of(page, size, Sort)
- Returns Page<T> with content, totalElements, totalPages, number

## Bulk Insert (Contact Import)

- Hibernate JDBC batching: hibernate.jdbc.batch\_size=500
- hibernate.order\_inserts=true groups INSERT statements for batch execution
- contactRepository.saveAll(contacts) triggers batch INSERT

# PART 12 - JPA / HIBERNATE / ORM

## How MailAlly Uses JPA Annotations

### @Entity + @Table

Every entity maps to a MySQL table. Example from `Contact.java`:

```
@Entity
@Table(name = "contacts")
public class Contact { ... }
```

### @Id + @GeneratedValue

Auto-increment primary keys:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

### @ManyToOne (Most used relationship)

```
// Contact belongs to Organization
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "organization_id", nullable = false)
private Organization organization;
```

### @PrePersist / @PreUpdate (Lifecycle callbacks)

Used across ALL entities for automatic timestamp management:

```
@PrePersist
protected void onCreate() {
    if (this.createdAt == null) this.createdAt = LocalDateTime.now();
    if (this.updatedAt == null) this.updatedAt = LocalDateTime.now();
    if (this.status == null) this.status = "SUBSCRIBED";
    if (this.isDeleted == null) this.isDeleted = false;
}

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}
```

### Lazy Loading

All `@ManyToOne` relationships use `FetchType.LAZY` - related entities are loaded only when accessed, avoiding unnecessary JOINS.

### Foreign Key Constraint Suppression

MailAlly deliberately suppresses database-level foreign key constraints:

```
@JoinColumn(name = "organization_id", foreignKey = @ForeignKey(ConstraintMode.NO_CONSTRAINT))
```

**Why?** This allows Hibernate `ddl-auto=update` to manage schema without FK conflicts, and enables flexible data operations during development. **Trade-off:** Referential integrity is enforced by application code, not the database.

### JDBC Batching Configuration

```
spring.jpa.properties.hibernate.jdbc.batch_size=500
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

This enables Hibernate to batch up to 500 INSERT/UPDATE statements into a single database round-trip, critical for contact import and campaign recipient processing.

# PART 13 - AUTHENTICATION & SECURITY

## Complete Security Architecture

### Registration Flow

```
Frontend RegisterPage -> POST /api/v1/auth/register
    v
AuthController.register(RegisterRequestDto)
    v
AuthServiceImpl.register():
  1. Validate email is not blank
  2. Check if user exists -> if yes, update password and return
  3. Find or create Subscription (default plan)
  4. Create Organization with unique slug
  5. Create User with role="ADMIN"
  6. Create Auth with BCrypt-hashed password
  7. Return UserProfileDto
```

### Login Flow

```
Frontend LoginPage -> POST /api/v1/auth/login
    v
AuthController.login(LoginRequestDto)
    v
AuthServiceImpl.login():
  1. Find user by email (case-insensitive)
  2. Find auth record for user
  3. BCrypt.matches(rawPassword, passwordHash)
  4. Verify user status is "ACTIVE"
  5. Update lastLoginAt timestamp
  6. Create CustomUserDetails (implements Spring UserDetails)
  7. JwtService.generateToken(userDetails) -> JWT with claims:
    { "userId": 1, "organizationId": 1, "role": "ADMIN",
      "sub": "admin@mailally.com", "iat": ..., "exp": +24h }
  8. Return AuthResponseDto { token, user }
```

### JWT Token Structure

- **Algorithm:** HMAC-SHA (HS256+)
- **Secret key:**  
MailAllySuperSecretKeyForJWTAuthentication2026ProductionGradeTokenGenerationKey
- **Expiration:** 24 hours (86,400,000 ms)
- **Claims:** userId, organizationId, role, subject (email)
- **Library:** jjwt 0.12.6

### Authenticated Request Flow

```
Every API request (except permitAll endpoints)
    v
JwtAuthenticationFilter.doFilterInternal():
  1. Extract "Authorization: Bearer <token>" header
  2. If missing -> pass to next filter (unauthenticated)
  3. JwtService.extractUsername(token) -> email
  4. CustomUserService.loadUserByUsername(email) -> UserDetails
  5. JwtService.isTokenValid(token, userDetails) -> check signature + expiration
  6. Create UsernamePasswordAuthenticationToken
  7. Set SecurityContextHolder.getContext().setAuthentication(authToken)
  8. Pass to controller
```

### Permitted (Public) Endpoints

```
/api/v1/auth/register
/api/v1/auth/login
/api/v1/auth/forgot-password
/api/v1/auth/reset-password
/api/v2/webhooks/**
/api/v1/webhooks/**
/api/integrations/google/callback
/ws-connect/**
/swagger-ui/**
/v3/api-docs/**
```

## Password Reset

```
POST /api/v1/auth/forgot-password { email }
-> Generates UUID reset token, sets 24h expiry
-> [NOTE: No email is actually sent - token is stored in DB only]
```

```
POST /api/v1/auth/reset-password { token, newPassword }
-> Finds auth by reset token, validates expiry
-> Updates passwordHash, clears token
```

**[PARTIALLY IMPLEMENTED]** - The forgot-password flow generates a token but does NOT send a reset email. The token must be retrieved from the database manually.

## CORS Configuration

```
config.setAllowedOriginPatterns(List.of(
    "http://localhost:5173", // Vite dev server
    "http://localhost:3000",
    "http://127.0.0.1:5173"
));
config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
config.setAllowCredentials(true);
```

## What's NOT Implemented

- [PLANNED / NOT IMPLEMENTED] **Refresh tokens** - Only access tokens exist; no token refresh mechanism
- [PLANNED / NOT IMPLEMENTED] **Email verification** - Registration does not require email verification
- [PLANNED / NOT IMPLEMENTED] **2FA/MFA** - Auth.mfaEnabled field exists but is always false
- [PLANNED / NOT IMPLEMENTED] **Role-based authorization** - Roles exist in User entity but no @PreAuthorize or role-based access control is enforced at the controller level

# PART 14 - USER / ROLE / PERMISSION ARCHITECTURE

## Roles (Stored as String in `users.role`)

- ADMIN - Full access (default role assigned at registration)
- MANAGER - [PARTIALLY IMPLEMENTED] - Validated in `EmailValidator.validateAdminOrManager()` but no distinct permissions defined
- VIEWER - [PLANNED / NOT IMPLEMENTED]

## Current Implementation

The role is stored as a string column and embedded in JWT claims. The `CustomUserDetails.getAuthorities()` returns `ROLE_<role>` but **no endpoint-level role enforcement exists**. Any authenticated user can access any endpoint.

```
// CustomUserDetails.java
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_" + role));
}
```

## What's Missing

- No `@PreAuthorize("hasRole('ADMIN')")` annotations on controllers
- No permission matrix implementation
- All authenticated users have full access to all endpoints
- [PARTIALLY IMPLEMENTED] - `EmailValidator.validateAdminOrManager()` checks role for email operations only

# PART 15 - MULTI-TENANCY

## Implementation: Organization-Level Row Filtering

MailAllly uses **shared database, shared schema** multi-tenancy where every data table has an `organization_id` column and all queries filter by the authenticated user's organization.

### How It Works

- **JWT carries organizationId:** When a user logs in, their `organizationId` is embedded in the JWT token
- **CustomUserDetails exposes it:** `currentUser.getOrganizationId()` returns the org ID
- **Every repository query filters by orgId:**

```
contactRepository.findByIdAndIsDeletedFalse(currentUser.getOrganizationId())
campaignRepository.findByIdAndOrganizationIdAndIsDeletedFalse(campaignId, orgId)
```

### Tenant Isolation Verification

```
Org A (id=1):
Users:      findByIdAndIsDeletedFalse(1)
Contacts:   findByIdAndIsDeletedFalse(1)
Campaigns:  findByIdAndIsDeletedFalse(1)
Templates:  findByIdAndIsDeletedFalse(1)

Org B (id=2):
-> Completely isolated - queries always filter by org ID
-> A user from Org A CANNOT see Org B data because
    their JWT contains organizationId=1
```

### Actual Enforcement Points

- `CampaignOrchestrator.verifyPreFlightChecks()` explicitly validates `campaign.getOrganization().getId().equals(organizationId)`
- Repository methods like `findByIdAndOrganizationIdAndIsDeletedFalse()` enforce tenant boundaries at the query level

### Weakness

- **No automatic Hibernate filter** - Each query must manually include `organizationId`. If a developer forgets to add the org filter, cross-tenant data leakage can occur.
- **No database-level Row Security** - Enforcement is purely at the application layer.

# PART 16 - CONTACT MANAGEMENT

## Manual Contact Creation

```
Frontend: contactApi.createContact(data)
      v
POST /api/v1/contacts { firstName, lastName, email, phone, company, ... }
      v
ContactController -> ContactServiceImpl.createContact()
      v
1. Validate organization exists
2. Validate email format
3. Check for duplicates within organization
4. Extract emailDomain from email address
5. Set sourceType = "MANUAL"
6. Set status = "SUBSCRIBED"
7. Save Contact entity
      v
Database INSERT into contacts table
```

## Bulk Import (CSV/Excel)

```
Frontend: contactApi.startImport(formData)  <- FormData with file + tag
      v
POST /api/v1/contacts/import/start (multipart/form-data)
      v
ContactController -> ContactServiceImpl:
1. Parse file type (CSV via Commons CSV, Excel via Apache POI)
2. Create ImportBatch record with batchCode
3. For each row:
  a. Map columns to Contact fields
  b. Validate email
  c. Check for duplicates (by email + organization)
  d. Build Contact entity
  e. Set sourceType = "CSV_IMPORT" or "EXCEL_IMPORT"
  f. Set importBatchId, importBatchName, importDate
4. Bulk save: contactRepository.saveAll(contacts) (JDBC batching)
5. Record import errors in ImportError table
6. Return import summary (total, success, failed, errors)
```

## Google Drive/Sheets Import

```
contactApi.importGoogleDriveFile(fileId, tag)
      v
POST /api/v1/integrations/google/import/drive
      v
GoogleIntegrationController -> GoogleDriveImportService:
1. Retrieve file from Google Drive API
2. Parse file content (CSV/Excel format)
3. Map to Contact entities
4. Bulk save with sourceType = "GOOGLE_DRIVE"
```

## Contact Features

- **Collections** - Group contacts into named collections (contact\_collections table)
- **Tags** - Comma-separated tags stored in contacts.tags column
- **Custom Fields** - JSON stored in contacts.custom\_fields (LONGTEXT)
- **Dynamic Field Registry** - Tracks custom field definitions per organization
- **Inline Cell Editing** - PATCH endpoint for single-field updates with audit history
- **Contact Timeline** - Activity feed per contact
- **Audit History** - Change tracking with old/new values
- **Saved Filters** - Persistent filter configurations
- **Domain Stats** - Aggregated stats by email domain



## PART 17 - TEMPLATE SYSTEM

### Template Entity (email\_templates table)

id, organization\_id, name, subject, html\_content (LONGTEXT), text\_content (LONGTEXT), status (DRAFT/ACTIVE/ARCHIVED), version (integer), created\_by, updated\_by

### Variable Personalization (TemplateRenderer.java)

Templates support {{variable}} placeholders that are replaced with contact data at send time:

```
// Supported variables:
{{firstName}}, {{lastName}}, {{email}}, {{phone}}, {{company}},
{{department}}, {{designation}}, {{city}}, {{state}}, {{country}},
{{address}}, {{postalCode}}, {{website}}
```

### How Rendering Works

```
Template: "Hello {{firstName}}, welcome to {{company}}!"
Contact: { firstName: "Ashok", company: "TechCorp" }
Result:   "Hello Ashok, welcome to TechCorp!"
```

### AI Template Generation

```
Frontend: templateApi.generateAiTemplate(data)
      v
POST /api/v1/templates/generate-ai
      v
TemplateController -> AiTemplateService:
  1. Send prompt to Gemini API
  2. Receive generated HTML content
  3. Return to frontend for review/editing
```

### Template Variable Engine (TemplateVariableEngine.java)

Separate service for listing available dynamic variables for the frontend template editor.

# PART 18 - CAMPAIGN ARCHITECTURE

## Complete Campaign Lifecycle

```
DRAFT -> VALIDATING -> PREPARING -> QUEUED -> RUNNING -> COMPLETED
|
+-- CANCELLED
+-- FAILED
+-- PAUSED
```

### Step 1: Campaign Creation

```
POST /api/v1/campaigns { name, subject, senderName, fromEmail }
-> Creates Campaign with status = "DRAFT"
```

### Step 2: Attach Template

```
POST /api/v1/campaigns/{id}/attach-template/{templateId}
-> Sets campaign.template_id
```

### Step 3: Add Audience (Collection)

```
POST /api/v1/campaigns/{id}/add-collection/{collectionId}
-> Links contacts from collection to campaign
```

### Step 4: Launch

#### Synchronous Launch (launchCampaign):

```
POST /api/v1/campaigns/{id}/launch
-> EmailServiceImpl.launchCampaign()
-> Sends emails one-by-one synchronously
-> Blocks until all emails are sent
```

#### Asynchronous Launch (launchCampaignAsync):

```
POST /api/v1/campaigns/{id}/launch (async mode)
-> CampaignOrchestrator.launchCampaign()
1. VALIDATING: Pre-flight checks (org match, sender email valid)
2. PREPARING: Resolve provider, prepare batches
3. BatchGenerator.generateAndQueueBatches()
   -> Creates CampaignBatch records
   -> Creates CampaignRecipientLog records (status=QUEUED)
   -> Pushes batch jobs to Redis Stream
4. QUEUED: Campaign enters the queue
5. CampaignWorkerService polls Redis Stream every 250ms
   -> Picks up batch jobs
   -> Processes in thread pool (4 concurrent workers)
   -> Uses Brevo Batch API for bulk sends
   -> Updates progress in Redis counters
6. COMPLETED: All batches processed
```

# PART 19 - EMAIL SENDING ENGINE

## Provider Architecture

```
EmailProviderFactory
+-- SmtplibEmailProvider    <- Spring JavaMailSender + Brevo SMTP relay
+-- BrevoEmailProvider      <- Brevo HTTP API v3 (batch + single sends)
+-- SesEmailProvider        <- AWS SES [STUB - credentials not configured]
+-- MockEmailProvider       <- Returns success without sending (testing)
```

## EmailProvider Interface

```
public interface EmailProvider {
    EmailSendResult send(String to, String toName, String from, String fromName,
                        String replyTo, String subject, String htmlBody);
    BatchSendResult sendBatch(List<RecipientBatchItem> recipients, ...);
    String getProviderName();
    boolean isAvailable();
}
```

## Provider Failover (EmailProviderFactory.sendWithFailover())

1. Send via primary provider (configured in application.properties)
2. If primary fails -> iterate through all registered providers
3. Try each backup provider that isAvailable()
4. Return first successful result
5. If ALL fail -> return original failure result

## Circuit Breaker (ProviderCircuitBreaker.java)

- Tracks provider success/failure counts
- On repeated failures -> opens circuit (blocks requests for cooldown period)
- Special handling for 429 (rate limit) vs 500 (server error)
- allowRequest(providerName) gate checked before each batch send

## Rate Limiting (RedisRateLimiter.java)

- Uses Redis to enforce per-provider send rate limits
- Prevents exceeding provider API quotas

# PART 20 - ASYNC / QUEUE / BATCH PROCESSING

## Technologies Used

- **Spring @Async** with `ThreadPoolTaskExecutor` - for async campaign launch
- **Virtual Threads** (Java 21) - `Executors.newVirtualThreadPerTaskExecutor()` for bulk email sends
- **Redis Streams** - distributed job queue for campaign batches
- **Fixed Thread Pool** - `CampaignWorkerService` uses `Executors.newFixedThreadPool(4)`

## Thread Pool Configuration (AsyncConfig.java)

```
@Bean(name = "emailTaskExecutor")
public Executor emailTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(5);        // 5 concurrent campaigns
    executor.setMaxPoolSize(20);        // burst to 20
    executor.setQueueCapacity(500);     // 500 queued campaigns
    executor.setThreadNamePrefix("MailAlly-Campaign-");
}
```

## Redis Streams Architecture

BatchGenerator -> Redis Stream "campaign:queue:pending" -> CampaignWorkerService

Stream message: { campaignId, batchId, provider }

```
CampaignWorkerService:
    @PostConstruct -> Create consumer group "campaign-workers-group"
    @Scheduled(fixedDelay=250) -> Poll stream every 250ms
    -> Acknowledge + delete message
    -> Submit processBatch() to thread pool
```

## Why Async Is Required

Sending 10,000 emails synchronously would block the HTTP thread for minutes. The async architecture:

- Returns HTTP 200 immediately with campaign status "QUEUED"
- Background workers process batches in parallel
- Redis counters provide real-time progress
- SSE/WebSocket streams push updates to frontend

# PART 21 - REDIS / CACHE / REAL-TIME STATE

## Redis Usage

### 1. Campaign Progress Counters (RedisProgressCache.java)

Key: campaign:{id}:progress:sent -> Integer counter  
Key: campaign:{id}:progress:failed -> Integer counter

Operations: INCREMENT (atomic)

Purpose: Real-time progress without querying database

Source of truth: Database (Redis is eventually synced by ProgressSyncScheduler)

### 2. Redis Streams (Job Queue)

Stream: campaign:queue:pending

Consumer Group: campaign-workers-group

Used by: BatchGenerator (producer) -> CampaignWorkerService (consumer)

Purpose: Distributed work queue for campaign batch processing

### 3. Rate Limiting (RedisRateLimiter.java)

Used for: Enforcing per-provider API rate limits

## Database vs Redis

| Concern           | Database (MySQL)     | Redis                              |
|-------------------|----------------------|------------------------------------|
| Source of truth   | [done] Yes           | [x] No (cache)                     |
| Campaign records  | [done] Persistent    | -                                  |
| Progress counters | Updated periodically | [done] Real-time atomic increments |
| Job queue         | -                    | [done] Redis Streams               |
| Rate limits       | -                    | [done] Sliding window              |

## ProgressSyncScheduler.java

Periodically reads Redis progress counters and writes them back to the campaigns table in MySQL, keeping the database in sync with Redis.

# PART 22 - SCHEDULER

## Implementation

- **Technology:** Spring Quartz with job-store-type=memory
- **Entity:** Scheduler entity with organization\_id, campaign\_id, scheduled\_at, status
- **Controller:** SchedulerController with CRUD + pause/resume endpoints
- **Service:** SchedulerServiceImpl

## Scheduled Campaign Flow

1. User schedules campaign via frontend
2. POST /api/v1/scheduler -> create Scheduler record (status=SCHEDULED)
3. Quartz triggers at scheduledAt time
4. SchedulerServiceImpl resolves campaign
5. Delegates to email engine for launch
6. Status: SCHEDULED -> RUNNING -> COMPLETED

## Limitations

- **In-memory job store** - Scheduled jobs are lost on application restart
- [PLANNED / NOT IMPLEMENTED] - JDBC job store for persistence across restarts
- No duplicate execution prevention beyond Quartz's internal mechanisms

# PART 23 - REAL-TIME CAMPAIGN MONITORING

## Two Mechanisms Implemented

### 1. Server-Sent Events (SSE) - Campaign Progress

```
GET /api/v1/emails/progress/{campaignId}/stream
  v
EmailServiceImpl.streamCampaignProgress()
  v
CampaignAsyncExecutor.createProgressEmitter(campaignId, campaign)
  v
SseEmitter sends periodic progress updates:
{ campaignId, sentCount, failedCount, totalRecipients, progressPercentage }
```

### 2. WebSocket (STOMP) - Live Analytics

```
WebSocketConfig.java:
  Registry endpoint: /ws-connect
  Message broker: /topic

WebhookController.broadcastUpdate():
  messagingTemplate.convertAndSend("/topic/analytics", eventData)

Frontend subscribes to /topic/analytics for real-time event push
(delivery, open, click, bounce events from webhooks)
```

# PART 24 - EMAIL TRACKING

## Email Event Types (EmailEventType.java)

```
public enum EmailEventType {  
    SENT,           // Email accepted by provider  
    DELIVERED,      // Email reached recipient inbox  
    OPENED,         // Recipient opened email (tracking pixel)  
    CLICKED,        // Recipient clicked a link  
    BOUNCED,        // Email bounced (hard/soft)  
    COMPLAINT,      // Recipient marked as spam  
    UNSUBSCRIBED    // Recipient unsubscribed  
}
```

## Webhook Processing Flow

```
External Provider (Brevo/SES)  
  v HTTP POST  
/api/v2/webhooks/brevo OR /api/v2/webhooks/ses  
  v  
WebhookController:  
1. Validate webhook signature (WebhookValidator)  
2. Normalize event (EventNormalizer) -> EmailEvent entity  
3. Idempotency check: existsByProviderMessageIdAndEventType()  
4. Resolve recipient: Find CampaignRecipientLog by providerMessageId  
5. Link event to campaign and recipient  
6. Update recipient status (e.g., SENT -> DELIVERED -> OPENED)  
7. Save EmailEvent record (immutable log)  
8. Broadcast via WebSocket (/topic/analytics)  
9. Publish to Kafka event bus (if enabled)
```

## Unresolved Webhooks

When a webhook arrives but the providerMessageId cannot be matched to a CampaignRecipientLog, the event is saved as an UnresolvedWebhookEvent for later retry resolution by WebhookResolverServiceImpl.



# PART 25 - ANALYTICS ARCHITECTURE

## Analytics Service (AnalyticsServiceImpl.java - 507 lines)

### Dashboard Overview

- Total campaigns, running/completed/failed counts
- Total contacts, active (SUBSCRIBED) contacts
- Total templates, segments
- Total emails sent, delivery rate
- System health status

### Campaign Analytics

- Per-campaign: sent, delivered, failed, bounced, opened, clicked
- Rates: delivery rate, open rate, click rate, bounce rate
- Uses EventAggregationEngine for real-time aggregation from email\_events table

### Report Export

- **CSV** - Apache Commons CSV writer
- **Excel** - Apache POI XSSFWorkbook
- **PDF** - Returns PdfReportDto (actual PDF rendering [PARTIALLY IMPLEMENTED])

### Time Series Data

- Daily, weekly, monthly, yearly data points
- [PARTIALLY IMPLEMENTED] - Currently generates sample/mock time-series data, not fully driven by real email event timestamps

# PART 26 - API ARCHITECTURE

## Complete API Catalog

### Authentication (/api/v1/auth)

| Method | Endpoint         | Purpose                |
|--------|------------------|------------------------|
| POST   | /register        | Register org + user    |
| POST   | /login           | Authenticate, get JWT  |
| POST   | /logout          | Clear security context |
| POST   | /change-password | Change password (auth) |
| POST   | /forgot-password | Generate reset token   |
| POST   | /reset-password  | Reset via token        |
| GET    | /profile         | Get user profile       |

### Contacts (/api/v1/contacts)

| Method | Endpoint                | Purpose                   |
|--------|-------------------------|---------------------------|
| GET    | /                       | List contacts (paginated) |
| POST   | /                       | Create contact            |
| GET    | / {id}                  | Get contact by ID         |
| PUT    | / {id}                  | Update contact            |
| DELETE | / {id}                  | Soft delete contact       |
| POST   | / {id}/restore          | Restore deleted contact   |
| POST   | / {id}/duplicate        | Duplicate contact         |
| PATCH  | / {id}/field            | Inline field update       |
| PATCH  | / {id}/cell             | Inline cell edit          |
| GET    | / {id}/history          | Contact audit history     |
| GET    | / {id}/timeline         | Contact timeline          |
| POST   | /import/start           | CSV/Excel import          |
| GET    | /import/progress/{code} | Import progress           |
| GET    | /import/history         | Import batch history      |
| GET    | /filter                 | Advanced filtering        |
| POST   | /bulk                   | Bulk actions              |
| GET    | /collections            | List collections          |
| POST   | /collections            | Create collection         |
| GET    | /fields                 | Dynamic field registry    |
| GET    | /tags                   | List tags                 |
| GET    | /filters/saved          | Saved filters             |
| GET    | /stats/domains          | Domain statistics         |

## Campaigns (/api/v1/campaigns)

| Method | Endpoint                        | Purpose                       |
|--------|---------------------------------|-------------------------------|
| GET    | /                               | List campaigns                |
| POST   | /                               | Create campaign               |
| GET    | /{{id}}                         | Get campaign                  |
| DELETE | /{{id}}                         | Delete campaign               |
| POST   | /{{id}}/attach-template/{{tid}} | Attach template               |
| POST   | /{{id}}/add-collection/{{cid}}  | Add contact collection        |
| POST   | /{{id}}/launch                  | Launch campaign               |
| POST   | /{{id}}/control?action=         | Control (PAUSE/RESUME/CANCEL) |
| GET    | /{{id}}/live-progress           | Real-time progress            |
| GET    | /{{id}}/failures                | Failed recipients             |
| GET    | /{{id}}/diagnostics             | Campaign diagnostics          |

## Email Engine (/api/v1/emails)

| Method | Endpoint                | Purpose                 |
|--------|-------------------------|-------------------------|
| POST   | /send                   | Send single email       |
| POST   | /send-bulk              | Send bulk emails        |
| POST   | /launch                 | Launch campaign (sync)  |
| POST   | /launch-async           | Launch campaign (async) |
| GET    | /progress/{{id}}/stream | SSE progress stream     |
| POST   | /retry/{{id}}           | Retry failed emails     |
| POST   | /cancel/{{id}}          | Cancel sending          |
| GET    | /status/{{id}}          | Email status            |
| GET    | /logs                   | Email logs (paginated)  |
| GET    | /stats/{{id}}           | Delivery statistics     |
| GET    | /queue/{{id}}           | Queue status            |
| GET    | /providers/health       | Provider health         |

## Webhooks (/api/v2/webhooks)

| Method | Endpoint | Purpose                |
|--------|----------|------------------------|
| GET    | /brevo   | Health check           |
| POST   | /brevo   | Brevo webhook receiver |
| POST   | /ses     | SES webhook receiver   |

## Analytics (/api/v1/analytics)

| Method | Endpoint        | Purpose                   |
|--------|-----------------|---------------------------|
| GET    | /dashboard      | Dashboard overview        |
| GET    | /campaigns      | Campaign analytics        |
| GET    | /campaigns/{id} | Single campaign analytics |
| GET    | /providers      | Provider analytics        |
| GET    | /charts         | Chart data                |
| GET    | /templates      | Template analytics        |
| GET    | /segments       | Segment analytics         |
| GET    | /scheduler      | Scheduler analytics       |
| GET    | /audience       | Audience analytics        |
| GET    | /export/csv     | CSV export                |
| GET    | /export/excel   | Excel export              |
| GET    | /export/pdf     | PDF export                |

## Other Modules

Templates, Segments, Scheduler, Notifications, Settings, Billing, Subscriptions, Audit, AI, Users, Organizations - all follow the same CRUD pattern at `/api/v1/{module}`.

## PART 27 - ERROR HANDLING

### GlobalExceptionHandler.java

| Exception                       | HTTP Status               | Handler                                          |
|---------------------------------|---------------------------|--------------------------------------------------|
| CustomException                 | 400 Bad Request           | Catches all business logic errors                |
| MethodArgumentNotValidException | 400 Bad Request           | Jakarta Validation failures (field-level errors) |
| HttpMessageNotReadableException | 400 Bad Request           | Malformed JSON                                   |
| BadCredentialsException         | 401 Unauthorized          | Wrong password                                   |
| UsernameNotFoundException       | 401 Unauthorized          | User not found                                   |
| Exception (catch-all)           | 500 Internal Server Error | Unhandled exceptions                             |

### Standard API Response Format

```
{
  "success": true/false,
  "message": "Human-readable message",
  "data": { ... } | null,
  "timestamp": "2026-08-10T23:00:00"
}
```

# PART 28 - CONFIGURATION & ENVIRONMENT

## application.properties - Key Settings

### Database

```
spring.datasource.url=jdbc:mysql://localhost:3306/mailally
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

### JWT

```
application.security.jwt.secret-key=MailAlly...Key # Warning: HARDCODED SECRET
application.security.jwt.expiration=86400000 # 24 hours
```

### Email Engine

```
mailally.email.active-provider=SMTP
mailally.email.default-sender-name=MailAlly
mailally.email.default-sender-email=info@marcamor.com
mailally.email.max-retries=3
```

### SMTP (Brevo Relay)

```
spring.mail.host=smtp-relay.brevo.com
spring.mail.port=587
spring.mail.username=b2fbd6001@smtp-brevo.com
spring.mail.password=xsmtpsib-... # Warning: HARDCODED API KEY
```

### Redis

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

### Kafka

```
spring.kafka.enabled=false # Disabled by default
```

### Warning: SECRETS THAT MUST NEVER BE COMMITTED

- JWT secret key
- SMTP password / Brevo API key
- SES access/secret keys
- Gemini AI API key
- Google OAuth client secret
- Google token encryption secret
- Database password

**All of these are currently HARDCODED in application.properties.**

## PART 29 - EXTERNAL SERVICES

| Service           | Purpose                               | Auth Method               | Status                           |
|-------------------|---------------------------------------|---------------------------|----------------------------------|
| Brevo SMTP Relay  | Primary email sending                 | SMTP credentials          | [done] Active                    |
| Brevo HTTP API    | Bulk batch sends, webhooks            | API key in header         | [done] Active                    |
| Amazon SES        | Cloud email sending                   | Access/Secret keys        | Warning: STUB (placeholder keys) |
| Google OAuth2     | Drive/Sheets access                   | OAuth2 client credentials | [done] Implemented               |
| Google Drive API  | List/download files for import        | OAuth2 bearer token       | [done] Implemented               |
| Google Sheets API | Read spreadsheet data for import      | OAuth2 bearer token       | [done] Implemented               |
| Google Gemini AI  | Content generation, spam scoring      | API key                   | [done] Implemented               |
| Redis             | Cache, job queue, rate limiting       | No auth (localhost)       | [done] Active                    |
| Kafka             | Event bus (analytics, audit, billing) | No auth (localhost)       | Warning: Disabled by default     |

# PART 30 - TESTING ARCHITECTURE

## Current State: MINIMAL

The project contains a test directory at mailally-backend/src/test/java/ but based on inspection:

- [PARTIALLY IMPLEMENTED] - Standard Spring Boot test class exists but no comprehensive test suite
- Dependencies present: spring-boot-starter-test, spring-security-test
- **No unit tests for services**
- **No integration tests for controllers**
- **No repository tests**
- **No end-to-end tests**
- Several PowerShell test scripts exist for manual SMTP testing (gate1-smtp-test-v5.ps1, test-brevo.ps1)

## What Should Be Tested

- AuthServiceImpl - Registration, login, password change
- ContactServiceImpl - CRUD, import, dedup logic
- EmailServiceImpl - Campaign launch, retry, cancel
- JwtService - Token generation, validation, expiration
- TemplateRenderer - Variable replacement
- WebhookController - Event normalization, idempotency



# PART 31 - DEPLOYMENT & DEVOPS

## Local Development

```
# Backend (requires Java 21, MySQL, Redis)
cd mailally-backend/mailally-backend
./mvnw spring-boot:run          # Port 8081

# Frontend (requires Node.js)
cd mailally-frontend
npm install
npm run dev                     # Port 5173 (Vite)
```

## Launch Scripts

- `start_app.bat / start_app.ps1` - Launch both frontend and backend
- `run_backend.bat` - Backend only
- `run_frontend.bat` - Frontend only
- `clean_project.bat` - Clean build artifacts

## Build

```
# Backend
./mvnw clean package -DskipTests    # Creates JAR in target/

# Frontend
npm run build                       # Creates dist/ folder
```

## What's NOT Implemented

- [PLANNED / NOT IMPLEMENTED] **Docker** - No Dockerfile or docker-compose.yml
- [PLANNED / NOT IMPLEMENTED] **CI/CD** - No GitHub Actions, Jenkins, or pipeline config
- [PLANNED / NOT IMPLEMENTED] **Cloud deployment** - No AWS/GCP/Azure config
- [PLANNED / NOT IMPLEMENTED] **HTTPS** - Local dev runs on HTTP only
- [PLANNED / NOT IMPLEMENTED] **Reverse proxy** - No Nginx/Traefik configuration
- [PLANNED / NOT IMPLEMENTED] **Monitoring dashboards** - Prometheus metrics endpoint exists but no Grafana setup
- [PLANNED / NOT IMPLEMENTED] **Database migrations** - Uses Hibernate auto-DDL; no Flyway/Liquibase

# PART 32 - COMPLETE APPLICATION DATA FLOWS

## 1. Registration

```
Browser -> POST /auth/register { email, password, organizationName }
-> AuthServiceImpl.register()
  -> Create Subscription (if none exists)
  -> Create Organization with unique slug
  -> Create User (role=ADMIN, status=ACTIVE)
  -> Create Auth (BCrypt hash)
-> Return UserProfileDto -> Frontend -> Redirect to /login
```

## 2. Login

```
Browser -> POST /auth/login { email, password }
-> AuthServiceImpl.login()
  -> Find User -> Find Auth -> BCrypt verify -> Generate JWT
-> Return { token, user } -> localStorage -> Redirect to /dashboard
```

## 3. Contact CSV Import

```
Browser -> POST /contacts/import/start (multipart form)
-> ContactServiceImpl
  -> Parse CSV (Commons CSV) or Excel (Apache POI)
  -> Create ImportBatch record
  -> For each row: validate, dedup, create Contact
  -> Bulk save (JDBC batching, batch_size=500)
  -> Record ImportErrors
-> Return ImportSummary
```

## 4. Campaign Launch (Async)

```
Browser -> POST /campaigns/{id}/launch
-> CampaignOrchestrator.launchCampaign()
  -> VALIDATING: tenant check, sender validation
  -> PREPARING: resolve provider
  -> BatchGenerator: create batches, create QUEUED recipients
  -> Push batch IDs to Redis Stream "campaign:queue:pending"
  -> QUEUED
-> CampaignWorkerService (background, polls every 250ms)
  -> Read from Redis Stream
  -> processBatch():
    -> Validate recipients (email format)
    -> TemplateRenderer: personalize subject + body
    -> BrevoEmailProvider.sendBatch() -> Brevo HTTP API
    -> Update CampaignRecipientLog: QUEUED -> ACCEPTED
    -> RedisProgressCache.incrementSent()
    -> Update CampaignBatch: PROCESSING -> COMPLETED
-> ProgressSyncScheduler: Redis counters -> MySQL campaign table
```

## 5. Webhook Event (Brevo Delivery)

```
Brevo -> POST /api/v2/webhooks/brevo { event: "delivered", message-id: "abc123" }
-> WebhookController.receiveBrevoWebhook()
  -> WebhookValidator.validateBrevoWebhook()
  -> WebhookResolverService.processWebhookEvent()
    -> Find CampaignRecipientLog by providerMessageId
    -> Create EmailEvent (type=DELIVERED)
    -> Update recipient status
    -> Save to database
-> broadcastUpdate() -> WebSocket /topic/analytics
```

## 6. Analytics Dashboard

```
Browser -> GET /analytics/dashboard
-> AnalyticsServiceImpl.getDashboardOverview()
  -> Count campaigns by status (from campaigns table)
  -> Count contacts by status (from contacts table)
  -> Count templates, segments
  -> Sum sentCount across campaigns
  -> Return DashboardOverviewDto
-> Frontend renders KPI cards + charts (Recharts)
```

## PART 33 - COMPLETE ARCHITECTURE DIAGRAM



## PART 34 - FILE-BY-FILE LEARNING MAP

### Security (Learn JWT + Spring Security)

| File                         | Purpose                    | Learn                                             |
|------------------------------|----------------------------|---------------------------------------------------|
| SecurityConfig.java          | Filter chain, CORS, BCrypt | How Spring Security protects endpoints            |
| JwtService.java              | Token create/validate      | How JWT works with HMAC signing                   |
| JwtAuthenticationFilter.java | Request interceptor        | How every request is authenticated                |
| CustomUserDetails.java       | User principal             | How Spring Security represents the logged-in user |

### Email Engine (Learn distributed email dispatch)

| File                        | Purpose                        | Learn                                        |
|-----------------------------|--------------------------------|----------------------------------------------|
| EmailProviderFactory.java   | Provider selection + failover  | Strategy pattern, fault tolerance            |
| SmtplibEmailProvider.java   | SMTP sending                   | How JavaMailSender works                     |
| BrevoEmailProvider.java     | HTTP API sending               | REST client, batch operations                |
| CampaignOrchestrator.java   | Launch state machine           | Orchestration pattern, pre-flight checks     |
| BatchGenerator.java         | Create batches + push to Redis | Queue-based work distribution                |
| CampaignWorkerService.java  | Process batches from queue     | Consumer pattern, Redis Streams, concurrency |
| TemplateRenderer.java       | Variable replacement           | Simple templating engine                     |
| ProviderCircuitBreaker.java | Failure detection              | Circuit breaker pattern                      |
| RedisProgressCache.java     | Fast counters                  | Redis atomic operations                      |
| WebhookController.java      | External event ingestion       | Webhook pattern, idempotency                 |

### Frontend (Learn React SPA patterns)

| File            | Purpose               | Learn                                   |
|-----------------|-----------------------|-----------------------------------------|
| App.jsx         | Root component tree   | Provider pattern (Auth, Theme, Toast)   |
| AppRoutes.jsx   | Route definitions     | React Router, protected routes          |
| AuthContext.jsx | Auth state management | Context API, localStorage, JWT handling |
| axiosClient.js  | API client            | Axios interceptors, token injection     |
| campaignApi.js  | Campaign API calls    | API abstraction pattern                 |

# PART 35 - HOW THE APPLICATION WAS BUILT

## Reverse-Engineered Development Sequence

### ACTUALLY IMPLEMENTED [done]

- **Project initialization** - Spring Boot 4.0.7 with Maven, Java 21
- **Database setup** - MySQL with Hibernate auto-DDL
- **Security** - JWT authentication, BCrypt, Spring Security filter chain
- **Organization & User** - Multi-tenant entities, registration flow
- **Auth module** - Login, logout, password management
- **Contact management** - CRUD, CSV/Excel import, collections, tags
- **Template system** - HTML templates with variable personalization
- **Segment management** - Contact segmentation with filters
- **Campaign module** - CRUD, template attachment, audience selection
- **Email engine (synchronous)** - Direct email sending via SMTP/Brevo
- **Email engine (async)** - Redis Streams, batch workers, orchestrator
- **Provider abstraction** - Factory pattern with failover + circuit breaker
- **Webhook processing** - Brevo and SES event receivers
- **Analytics** - Dashboard overview, campaign metrics, exports
- **Dashboard** - Executive KPIs, system health
- **Scheduler** - Quartz-based campaign scheduling
- **Notifications** - In-app notification system
- **Billing** - Basic billing records and subscription management
- **Audit** - Activity logging
- **AI** - Gemini-powered content generation
- **Google Integration** - OAuth2, Drive, Sheets contact import
- **Settings** - Application configuration management
- **Frontend SPA** - React 19 with all 15+ pages
- **Real-time** - WebSocket + SSE for live updates
- **Default data seeding** - Admin users created on startup

### PLANNED BUT NOT IMPLEMENTED [x]

- Docker containerization
- CI/CD pipeline
- Comprehensive test suite
- Email verification on registration
- Refresh tokens
- Role-based access control enforcement
- Payment gateway integration
- PDF report generation (actual rendering)

- Database migrations (Flyway/Liquibase)
- HTTPS/TLS configuration
- Production cloud deployment
- Monitoring dashboards (Grafana)

# PART 36 - WHAT I NEED TO LEARN

## A. Frontend

| Concept               | Why MailAlly Needs It                         | Actual Files                      |
|-----------------------|-----------------------------------------------|-----------------------------------|
| React Context API     | Auth state, theme state across all components | AuthContext.jsx, ThemeContext.jsx |
| React Router          | Client-side page navigation, protected routes | AppRoutes.jsx                     |
| Axios Interceptors    | Auto-attach JWT, handle 401 redirects         | axiosClient.js                    |
| Component Composition | Layout + pages + shared components            | MainLayout.jsx, all pages         |

## B. Backend

| Concept                      | Why MailAlly Needs It                             | Actual Files                                      |
|------------------------------|---------------------------------------------------|---------------------------------------------------|
| Spring Security Filter Chain | Every request must be authenticated               | SecurityConfig.java, JwtAuthenticationFilter.java |
| JWT Authentication           | Stateless auth without sessions                   | JwtService.java                                   |
| Dependency Injection         | All services receive dependencies via constructor | Every service class                               |
| @Transactional               | Database operations must be atomic                | AuthServiceImpl.java, EmailServiceImpl.java       |
| Strategy Pattern             | Multiple email providers with same interface      | EmailProvider, EmailProviderFactory               |
| Builder Pattern              | Complex object construction                       | All entities with .builder()                      |

## C. Database

| Concept            | Why MailAlly Needs It               | Actual Files                            |
|--------------------|-------------------------------------|-----------------------------------------|
| JPA Entity Mapping | Java objects -> database rows       | All entity/ classes                     |
| Soft Deletion      | Never lose data, enable restore     | is_deleted column in every entity       |
| JDBC Batching      | Performance for bulk contact import | application.properties (batch_size=500) |

## D. Distributed Systems

| Concept         | Why MailAlly Needs It                                  | Actual Files                                    |
|-----------------|--------------------------------------------------------|-------------------------------------------------|
| Redis Streams   | Distributed job queue for campaign workers             | CampaignWorkerService.java, BatchGenerator.java |
| Circuit Breaker | Prevent cascading failures to email providers          | ProviderCircuitBreaker.java                     |
| Idempotency     | Prevent duplicate emails, duplicate webhook processing | CampaignBatch.idempotencyKey, WebhookController |

## E. Email Systems

| Concept                  | Why MailAlly Needs It               | Actual Files            |
|--------------------------|-------------------------------------|-------------------------|
| SMTP Protocol            | Core email transport                | SmtEmailProvider.java   |
| Provider API Integration | Brevo batch sends                   | BrevoEmailProvider.java |
| Webhook Processing       | Track delivery/opens/clicks/bounces | WebhookController.java  |
| Email Personalization    | {{firstName}} in templates          | TemplateRenderer.java   |

# PART 37 - BEGINNER -> PROFESSIONAL -> ARCHITECT

## JWT Authentication

**BEGINNER:** JWT is like a digital ID card. When you log in, the server gives you a signed token. You show this token with every request to prove who you are. The server doesn't need to remember your session.

**DEVELOPER:** MailAlly's `JwtService` uses HMAC-SHA signing via the `jjwt` library. The token payload contains `userId`, `organizationId`, and `role` claims. The `JwtAuthenticationFilter` intercepts every request, extracts the Bearer token, validates the signature and expiration, then loads the user via `CustomUserDetailsService` and sets the Spring `SecurityContext`.

**ARCHITECT:** The current implementation has a single HMAC key shared across all instances, which means horizontal scaling requires all instances to share the same secret. There are no refresh tokens, so the 24-hour access token must be re-obtained via login. Token revocation (logout) only clears the `SecurityContext` in-memory - a stolen token remains valid until expiration. For production: consider RSA key pairs (asymmetric signing), short-lived access tokens (15 min) with refresh token rotation, and a Redis token blacklist for revocation.

## Multi-Provider Email Engine

**BEGINNER:** MailAlly can send emails through different email services (like using different post offices). If one service is down, it automatically tries another one.

**DEVELOPER:** `EmailProviderFactory` uses the Strategy pattern. `EmailProvider` is the interface, with concrete implementations: `SmtplibEmailProvider`, `BrevoEmailProvider`, `SesEmailProvider`, `MockEmailProvider`. The `sendWithFailover()` method tries the primary provider first, then iterates through backups. `ProviderCircuitBreaker` tracks failures and blocks requests to unhealthy providers.

**ARCHITECT:** The failover is synchronous and single-threaded per email. For true resilience, consider: dead letter queues for permanently failed messages, provider health scoring (not just boolean availability), weighted provider routing based on cost/latency/reputation, and provider-specific rate limit awareness. The circuit breaker uses in-memory state, which doesn't share across multiple application instances - for multi-node deployment, this should use Redis-backed circuit breaking.



# PART 38 - HOW TO EXPLAIN MAILALLY TO AN EXPERT

## 5-Minute Explanation

"MailAlly is a multi-tenant SaaS email marketing platform. The frontend is a React SPA communicating with a Spring Boot REST API. Authentication is stateless JWT. Organizations are isolated by `organization_id` filtering. The email engine supports multiple providers (SMTP, Brevo API, SES) with automatic failover and circuit breaking. Campaign execution uses Redis Streams for job queuing with background workers processing batches in parallel. Webhook endpoints receive delivery events from providers. Analytics are aggregated from an immutable event log."

## 15-Minute Explanation

Add to above:

- "Contacts are managed with CSV/Excel/Google Sheets import using Apache POI and Commons CSV. Templates support `{{variable}}` personalization rendered at send time.
- Campaign lifecycle: DRAFT -> VALIDATING -> PREPARING -> QUEUED -> RUNNING -> COMPLETED. The orchestrator handles pre-flight checks, the batch generator creates work units, Redis Streams distribute them to workers, and workers use the Brevo batch API for efficient dispatch.
- Progress tracking uses Redis atomic counters for real-time updates, synced to MySQL periodically. WebSocket (STOMP) and SSE provide live monitoring.
- The analytics engine aggregates from `email_events` (immutable log) and `campaign_recipient_logs` for per-campaign metrics."

## 30-Minute Technical Deep Dive

Add to above: Entity relationship details, JPA configuration, Hibernate batching, security filter chain walkthrough, actual provider implementation differences, Redis Stream consumer group mechanics, circuit breaker state transitions, webhook idempotency handling, unresolved event retry mechanism, Google OAuth2 integration flow, AI template generation pipeline, and current architectural weaknesses with improvement paths.

## PART 39 - EXPERT QUESTIONS I SHOULD BE READY FOR

| Question                             | Answer Summary                                                                                                                                                                                                                                         |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why Spring Boot?                     | Mature ecosystem, dependency injection, JPA integration, security framework, extensive middleware support. v4.0.7 with Java 21 for virtual threads.                                                                                                    |
| Why MySQL?                           | Relational data with foreign key relationships, ACID transactions, mature tooling. Used with Hibernate auto-DDL for rapid development.                                                                                                                 |
| Why JPA/Hibernate?                   | Object-relational mapping eliminates manual SQL, provides entity lifecycle management, lazy loading, and JDBC batching.                                                                                                                                |
| Why DTOs?                            | Decouple internal entity structure from API contracts. Prevents exposing database columns, enables versioned APIs.                                                                                                                                     |
| Why Redis?                           | Sub-millisecond atomic counters for campaign progress (vs. database writes for every email sent). Redis Streams for distributed job queuing.                                                                                                           |
| Why asynchronous processing?         | Sending 10,000 emails synchronously would block the HTTP thread for minutes. Background workers return immediately and process in parallel.                                                                                                            |
| How does campaign batching work?     | BatchGenerator splits contacts into chunks (100 per batch), creates CampaignBatch records, pushes job payloads to Redis Stream. Workers poll the stream every 250ms, process one batch per thread, up to 4 concurrent workers.                         |
| How do you handle retries?           | Email.retryCount / maxRetries (default 3). CampaignBatch.retryCount with re-queue to Redis Stream. Circuit breaker gates retry attempts.                                                                                                               |
| How do you prevent duplicate emails? | CampaignBatch.idempotencyKey (UUID). Batch status check: skip if already COMPLETED or ACCEPTED. EmailEvent dedup: existsByProviderMessageIdAndEventType( ).                                                                                            |
| How do you handle provider failures? | EmailProviderFactory.sendWithFailover( ) tries backup providers. ProviderCircuitBreaker blocks requests to failing providers. 429 vs 500 differentiation for retry strategy.                                                                           |
| How do you isolate organizations?    | organization_id column on every data table. All repository queries filter by org ID from JWT. CampaignOrchestrator explicitly validates org ownership.                                                                                                 |
| How would you scale to 1M emails?    | Increase worker concurrency, add more worker nodes (Redis Streams support consumer groups), use Brevo batch API (current), add database sharding or read replicas, replace Quartz in-memory with JDBC store.                                           |
| What happens if Redis fails?         | Campaign progress tracking degrades (no real-time counters). Job queue becomes unavailable - async campaigns cannot be processed. Sync launch still works (bypasses Redis).                                                                            |
| What happens if the database fails?  | Complete system failure - all operations depend on MySQL. No read replica or failover configured.                                                                                                                                                      |
| What would you redesign?             | Add environment variable-based secrets, implement proper role-based access control, add database migrations (Flyway), implement refresh tokens, add comprehensive tests, containerize with Docker, add CI/CD, move to event-sourcing for email events. |

# PART 40 - ARCHITECTURAL WEAKNESSES

## Security Issues

| Problem                 | Current Implementation                                                | Risk                                              | Improvement                                  |
|-------------------------|-----------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------|
| Hardcoded secrets       | JWT key, API keys, DB password in <code>application.properties</code> | Credentials exposed in version control            | Use environment variables or secrets manager |
| No role enforcement     | Roles stored but not checked at endpoints                             | Any authenticated user can access admin endpoints | Add <code>@PreAuthorize</code> annotations   |
| No refresh tokens       | Single 24h access token                                               | Users must re-login; no way to revoke access      | Implement access + refresh token pair        |
| No email verification   | Registration succeeds without verifying email                         | Fake accounts, email spoofing                     | Add email verification flow                  |
| Frontend dev fallback   | <code>AuthContext.jsx</code> creates mock tokens on backend failure   | Bypasses authentication in development            | Remove fallback in production builds         |
| FK constraints disabled | <code>@ForeignKey(ConstraintMode.NO_CONSTRAINT)</code>                | Database can have orphaned records                | Enable FK constraints, use Flyway migrations |

## Performance Issues

| Problem              | Current Implementation                                                                 | Risk                                                                                    | Improvement                                           |
|----------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------|
| Sync campaign launch | <code>launchCampaign()</code> sends emails one-by-one on HTTP thread                   | Request timeout for large campaigns                                                     | Always use async launch path                          |
| N+1 queries          | <code>findAll().stream().filter()</code> in startup repair                             | Loads ALL records to filter in Java memory                                              | Use JPA <code>@Query</code> with WHERE clause         |
| No database indexes  | Hibernate auto-DDL doesn't create custom indexes                                       | Slow queries on <code>organization_id</code> , <code>status</code> , <code>email</code> | Add <code>@Index</code> annotations or SQL migrations |
| Full table scans     | <code>resetPassword</code> does <code>findAll().stream().filter()</code> to find token | Scans entire auth table                                                                 | Add <code>findByResetToken()</code> repository method |

## Scalability Issues

| Problem                      | Risk                                        | Improvement                                             |
|------------------------------|---------------------------------------------|---------------------------------------------------------|
| In-memory Quartz             | Scheduled jobs lost on restart              | Use JDBC job store                                      |
| Single-node workers          | Campaign processing limited to one JVM      | Deploy multiple worker nodes with Redis consumer groups |
| Monolith deployment          | Cannot scale email engine independently     | Extract email engine as separate microservice           |
| No connection pooling config | Default pool may be insufficient under load | Configure HikariCP pool size                            |

## Multi-Tenancy Risks

| Problem              | Risk                                                      | Improvement                                                                      |
|----------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------|
| Manual org filtering | Developer can forget <code>organizationId</code> in query | Use Hibernate <code>@Filter</code> or interceptor for automatic tenant filtering |
| No cross-tenant test | No automated test verifying tenant isolation              | Write integration tests that verify Org A cannot see Org B data                  |

# PART 41 - PRACTICAL CODE-TRACING EXERCISES

## Exercise 1: Trace Login

**Start at:** `mailally-frontend/src/pages/auth/LoginPage.jsx`

- Find the form submit handler
- Trace to the API call in `authApi.js`
- Find the HTTP endpoint and method
- Go to `AuthController.java` - find the matching `@PostMapping`
- Follow into `AuthServiceImpl.login()`
- Identify: Which repository finds the user? How is the password verified?
- How is the JWT token created? What claims does it contain?
- Trace the response back to the frontend - where is the token stored?

## Exercise 2: Trace Contact CSV Import

**Start at:** `contactApi.startImport()` in `contactApi.js`

- What HTTP method and content type is used?
- Find the controller endpoint in `ContactController.java`
- Follow into `ContactServiceImpl`
- How is the file parsed? (Hint: look for `CSVFormat` and `XSSFWorkbook`)
- How are duplicate contacts detected?
- How does JDBC batching work? (Check `application.properties`)

## Exercise 3: Trace Campaign Launch (Async)

**Start at:** `campaignApi.launchCampaign()` in `campaignApi.js`

- Find the controller method
- Follow into `CampaignOrchestrator.launchCampaign()`
- What validation happens in `verifyPreFlightChecks()`?
- How does `BatchGenerator.generateAndQueueBatches()` push jobs to Redis?
- In `CampaignWorkerService`, how often does `pollQueue()` run?
- How does `processBatch()` handle a successful Brevo batch API response?
- What happens when the circuit breaker is OPEN?

## Exercise 4: Trace a Brevo Webhook

**Start at:** `POST /api/v2/webhooks/brevo`

- Find `WebhookController.receiveBrevoWebhook()`
- How is the webhook signature validated?
- What does `WebhookResolverService.processWebhookEvent()` do?
- How is the `CampaignRecipientLog` found from the webhook payload?
- What happens if the provider message ID doesn't match any recipient?

# PART 42 - FINAL "I UNDERSTAND MAILALLY" CHECKLIST

## FRONTEND

- ☐ I understand how `App.jsx` sets up providers (Auth, Theme, Toast)
- ☐ I understand how `AppRoutes.jsx` defines protected vs public routes
- ☐ I understand how `AuthContext.jsx` manages login state with `localStorage`
- ☐ I understand how `axiosClient.js` attaches JWT tokens to every request
- ☐ I understand how API modules (`campaignApi.js`, etc.) abstract backend calls
- ☐ I understand the page -> component -> API call -> response -> state update cycle

## BACKEND

- ☐ I understand the feature-based package structure (controller/dto/entity/service/repository)
- ☐ I understand how `SecurityConfig.java` defines the filter chain
- ☐ I understand how `JwtAuthenticationFilter` intercepts and validates every request
- ☐ I understand how `AuthServiceImpl` handles registration and login
- ☐ I understand how `CustomUserDetails` carries `userId`, `organizationId`, `role`
- ☐ I understand how controllers use `@AuthenticationPrincipal` to get the current user
- ☐ I understand how `ApiResponse<T>` wraps all API responses
- ☐ I understand how `GlobalExceptionHandler` converts exceptions to HTTP responses

## DATABASE

- ☐ I understand the entity -> table mapping with JPA annotations
- ☐ I understand `@ManyToOne` relationships (Contact -> Organization, Campaign -> Template)
- ☐ I understand `@PrePersist` / `@PreUpdate` for automatic timestamps
- ☐ I understand soft deletion with `is_deleted` column
- ☐ I understand JDBC batching for bulk operations
- ☐ I understand why `FetchType.LAZY` is used on all relationships

## EMAIL ENGINE

- ☐ I understand `EmailProviderFactory` and the Strategy pattern
- ☐ I understand `sendWithFailover()` - try primary, then backups
- ☐ I understand `ProviderCircuitBreaker` - failure tracking and circuit states
- ☐ I understand `CampaignOrchestrator` - the campaign state machine
- ☐ I understand `BatchGenerator` - creating batches and pushing to Redis Stream
- ☐ I understand `CampaignWorkerService` - polling Redis, processing batches
- ☐ I understand `TemplateRenderer` - `{{variable}}` replacement
- ☐ I understand `RedisProgressCache` - atomic counters for real-time progress

## WEBHOOKS & TRACKING

- ☐ I understand how `WebhookController` receives events from Brevo/SES

- ☐ I understand EmailEventType enum (SENT, DELIVERED, OPENED, CLICKED, BOUNCED, COMPLAINT)
- ☐ I understand idempotency - existsByProviderMessageIdAndEventType()
- ☐ I understand how events are linked to campaigns via providerMessageId

## INFRASTRUCTURE

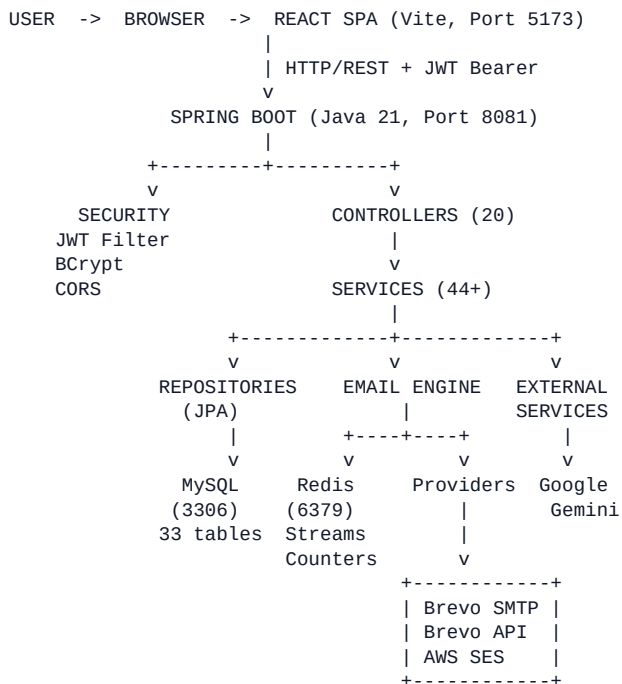
- ☐ I understand Redis usage: counters, streams, rate limiting
- ☐ I understand Quartz scheduler with in-memory job store
- ☐ I understand WebSocket (STOMP) for live analytics push
- ☐ I understand SSE for campaign progress streaming
- ☐ I understand Google OAuth2 integration for Drive/Sheets

## SYSTEM DESIGN

- ☐ I understand multi-tenancy via organization\_id filtering
- ☐ I understand async processing (why synchronous email would fail)
- ☐ I understand the circuit breaker pattern (why it exists)
- ☐ I understand retry mechanisms (email retries, batch retries)
- ☐ I understand the provider abstraction (why swap providers easily)

# PART 43 - FINAL ONE-PAGE ARCHITECTURE SUMMARY

## MAILALLY ARCHITECTURE



## CAMPAIGN EMAIL FLOW

```
CAMPAIGN (DRAFT)
  v User clicks Launch
ORCHESTRATOR (VALIDATING -> PREPARING)
  v Pre-flight checks pass
BATCH GENERATOR (Creates batches)
  v Push to Redis Stream
WORKER SERVICE (Polls every 250ms)
  v Process batch
TEMPLATE RENDERER (Personalize {{vars}})
  v
PROVIDER FACTORY (Select provider + failover)
  v
EMAIL PROVIDER (Brevo API / SMTP / SES)
  v Email delivered
WEBHOOK (Brevo/SES -> /api/v2/webhooks)
  v Normalize event
EMAIL EVENT (Immutable log: DELIVERED/OPENED/CLICKED)
  v
ANALYTICS ENGINE (Aggregate metrics)
  v
DASHBOARD (Frontend charts + KPIs)
```

## MULTI-TENANCY

```
Organization A --+
  Users           | Every query includes
  Contacts        +--> organization_id from JWT
  Campaigns       | = Complete data isolation
  Templates       |
Organization B --+
```

## KEY PATTERNS

```
- Strategy Pattern    -> EmailProvider interface
- Factory Pattern     -> EmailProviderFactory
- Circuit Breaker     -> ProviderCircuitBreaker
- Builder Pattern     -> All entities
- Provider Pattern    -> React AuthContext
- Interceptor Pattern -> Axios JWT injection
- Observer Pattern    -> WebSocket event push
- Queue Pattern       -> Redis Streams
```

- Soft Delete -> is\_deleted on all entities
- DTO Pattern -> Request/Response separation

=====

|                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i><b>This document was generated by analyzing the actual MailAlly codebase. No implementation was invented. All class names, file paths, endpoints, and configurations referenced are real and exist in the repository.</b></i> |
| <i><b>Last analyzed:</b> August 2026</i>                                                                                                                                                                                         |
| <i><b>Total backend entities:</b> 33</i>                                                                                                                                                                                         |
| <i><b>Total controllers:</b> 20</i>                                                                                                                                                                                              |
| <i><b>Total services:</b> 44+</i>                                                                                                                                                                                                |
| <i><b>Total frontend pages:</b> 15+</i>                                                                                                                                                                                          |
| <i><b>Total API endpoints:</b> 100+</i>                                                                                                                                                                                          |